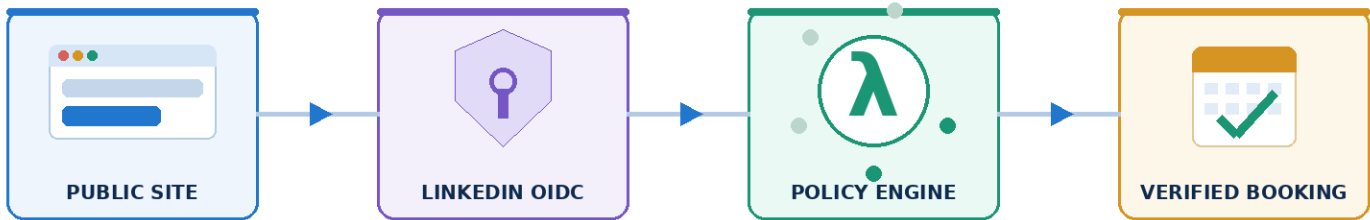


Recruiter Verification Gateway

An identity-gated, serverless scheduling system that verifies who is on the other side of the screen before it gives away time on the calendar — built on AWS Lambda, API Gateway, S3 and CloudFront, fronted by LinkedIn OpenID Connect, and shipped with an AI-generated avatar video resume.

REQUEST FLOW

anonymous click → federated identity → policy adjudication → verified booking



PASS FAIL UNAVAILABLE

162 / 162 POLICY COMBINATIONS VERIFIED

Esmeralda Cabrera Ventura

Solutions Architect & Founder, Apex Intelligent Systems

U.S. Air Force Veteran · M.S. Computer Science · Cloud · AI/ML · Automation

Every screenshot is an authentic capture from the real build. All account numbers, resource identifiers, endpoints, API keys, client secrets, signing keys and OAuth tokens have been permanently redacted.

PLATFORMS & SERVICES USED

AWS Console Lambda API Gateway S3 CloudFront CloudWatch IAM LinkedIn Dev OpenID Connect Calendly Google Calendar Node.js HTML-CSS-JS AI Avatar

Service icons above are original illustrative glyphs, not official brand marks.

Table of Contents

1. Executive Summary	4
2. Problem, Objectives & Significance	5
2.1 The Problem	5
2.2 Objectives	5
2.3 Why It Matters	5
3. System Architecture	6
3.1 Component Responsibilities	6
3.2 The Request Lifecycle	7
4. Technology Stack	7
5. The Verification Engine — Technical Deep Dive	9
5.1 The Failure That Shaped the Design	9
5.2 The Three-State Model	9
5.3 The Five Tests	9
5.4 The Policy Engine	10
5.5 Exhaustive Verification	10
5.6 The Signed Hand-off Token	10
5.7 Honest Assessment of the Hand-off	11
5.8 Failure Taxonomy	11
6. Implementation Walkthrough	12
6.1 Build Artifacts	12
6.2 The Protected Resource — Scheduling Layer	12
6.3 Identity Provider Registration	16
6.4 The Managed Front Door — API Gateway	18
6.5 Binding the Redirect URI	26
6.6 The Verification Function	28
6.7 First Live Test — Total Failure	32
6.8 Root-Cause Analysis	34
6.9 Static Hosting — Private Origin	37
6.10 Global Delivery — CloudFront	43
6.11 Origin Access Policy	49
6.12 Wiring the Outcome Routing	51
6.13 End-to-End Validation on the Live System	56

7. Results & Demonstrated Capabilities	66
7.1 Key Outcomes	66
7.2 Operational Characteristics	66
8. Security & Responsible Engineering	66
8.1 Disclosed Limitations	67
9. The AI Avatar Video Resume	67
10. Skills Demonstrated	68
11. Conclusion & Future Enhancements	69
11.1 What This Project Demonstrates	69
11.2 Planned Enhancements	70
12. Let's Build Something	70

Section 6 documents the entire build in the order it happened — provisioning, integration, total failure, diagnosis, correction and live validation — with 56 captioned screenshots, each accompanied by an explanation of what is happening and why that step exists.

1. Executive Summary

This project delivers a production system that answers a deceptively simple question: **how do you publish a 'Schedule an Interview' button on the public internet without handing your calendar to every bot, spammer and time-waster who finds it?**

The answer built here is a serverless identity gate. A visitor who wants time on the calendar must first authenticate through LinkedIn using OpenID Connect. Their identity assertion returns to an Amazon API Gateway HTTP endpoint, which invokes an AWS Lambda function that runs a five-test verification policy against the claims LinkedIn actually released. Approved visitors receive a cryptographically signed, short-lived token and are forwarded to a live booking calendar. Everyone else is redirected to a failure page that tells them precisely which criterion they did not meet. No server is provisioned, patched or paid for while idle.

The system is deployed behind a global CloudFront distribution serving a private S3 origin through Origin Access Control, so the storage bucket is never publicly addressable. The front end is a hand-built responsive portfolio; the back end is a single purpose-written Node.js function with zero third-party dependencies.

What makes this project worth reading is not that it works. It is *how it came to work*.

The first deployment failed. Not partially — **completely**. Every single visitor was rejected. The original logic treated LinkedIn's HTTP 403 responses on privileged endpoints as evidence that the visitor had failed verification, when those responses meant only that the application had not been granted access to that data. Absence of evidence was scored as evidence of absence, and the gate slammed shut on everybody. Compounding it, the function was provisioned with defaults that could not survive its own cold start.

Diagnosing that required instrumentation discipline: reproducing the failure live, reading the rendered page source, interrogating the browser DOM console, and finally going to CloudWatch — the only authoritative account of what the function actually did. The logs showed a three-second timeout on a runtime needing far longer to initialise, and a memory ceiling the function was consuming eighty percent of. Both were corrected at the root cause rather than patched around.

The engine was rebuilt around a **three-state model** — every test returns PASS, FAIL or UNAVAILABLE — so missing data can never be mistaken for a failed check. The resulting policy was verified exhaustively against all 162 valid combinations of five tests across three states, with a 162/162 pass rate. The approval hand-off was hardened with an HMAC-SHA256 signed token carrying a fifteen-minute expiry, a unique replay identifier and constant-time signature comparison.

Shipped alongside the platform is an **AI-generated avatar video resume** — a synthetic presenter built on the author's own likeness and voice, embedded directly in the portfolio. It is not a gimmick: it is a deliberate demonstration of applied generative AI, produced to prove the same capability the author sells.

2. Problem, Objectives & Significance

2.1 The Problem

A public portfolio has to solve two goals that pull directly against each other. It must be *maximally reachable* — a recruiter should be able to book time in seconds. And it must be *defended* — because a naked scheduling link on a public website is an open invitation to automated form-fillers, cold-outreach spam and calendar-stuffing bots.

The usual compromises are all bad. Hiding the link behind a contact form adds friction and loses the people you want. Publishing the raw calendar surrenders control. A CAPTCHA proves the visitor is human but says nothing about whether they are relevant — a bot and a genuine hiring manager are indistinguishable to a checkbox.

The real requirement is not proof of humanity. It is **proof of professional identity**: a verifiable claim, issued by a trusted third party, that the person requesting time is who they say they are.

2.2 Objectives

- Gate a public scheduling action behind a federated identity assertion rather than a human-detection puzzle.
- Implement verification as a stateless, serverless function with no persistent infrastructure to run, patch, scale or pay for at idle.
- Express admission rules as an explicit, auditable policy changed by configuration rather than by rewriting logic.
- Distinguish rigorously between a visitor who fails a check and a data source that declines to answer.
- Issue approval as a signed, expiring, replay-resistant credential.
- Give every rejected visitor a specific, actionable reason instead of a generic error.
- Serve the front end from a global CDN over a private origin.
- Instrument every decision so the live system can be reasoned about from logs alone.

2.3 Why It Matters

This is a small system that exercises a disproportionately large share of the modern cloud engineering surface: a full OAuth 2.0 / OpenID Connect authorization-code flow with CSRF defence, an event-driven serverless backend behind a managed HTTP front door, a CDN-fronted distribution with private origin access, applied cryptography, production observability, and a policy engine validated by exhaustive state-space enumeration.

Above all it demonstrates the discipline separating something that demos from something that *ships*: the willingness to discover that your system rejects one hundred percent of real users, instrument until you know exactly why, and fix the model rather than the symptom.

3. System Architecture

The architecture is a serverless request pipeline. There is no application server, no container, no virtual machine and no database. A static front end initiates a federated identity flow; a managed HTTP endpoint receives the callback; a single stateless function adjudicates it; and the outcome is expressed entirely as an HTTP redirect carrying either a signed approval token or a machine-readable rejection reason.

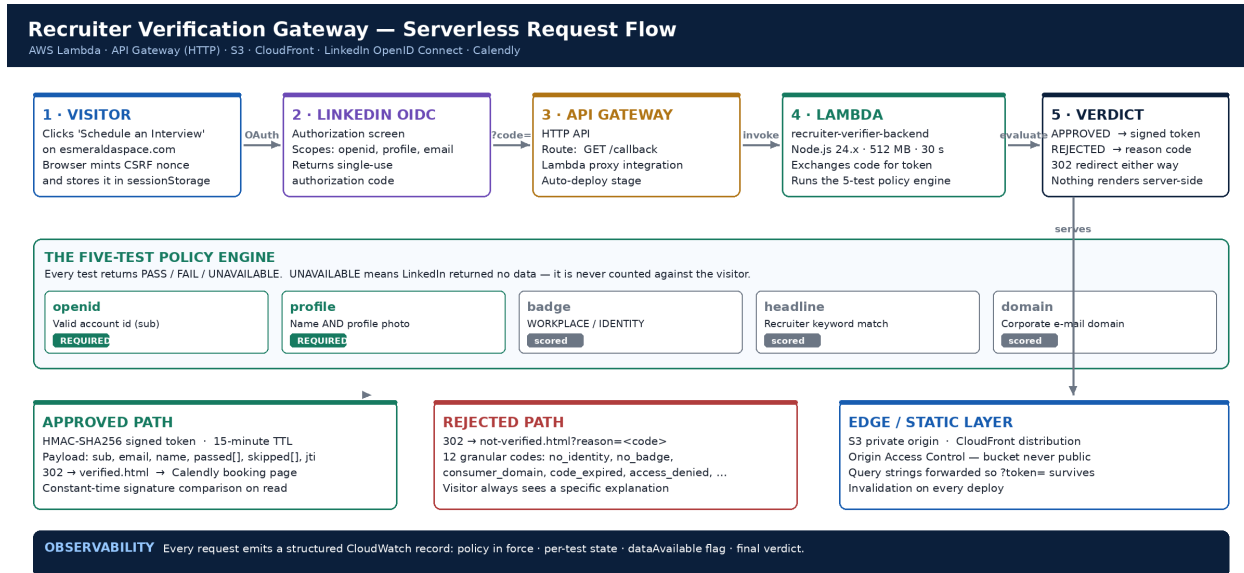


Figure 1. End-to-end architecture — from a click on the public site, through federated identity and the five-test policy engine, to either a signed booking token or a specific rejection reason.

3.1 Component Responsibilities

Component	Responsibility
Static front end (S3 + CloudFront)	Hosts the portfolio and the Schedule an Interview control. Mints a cryptographically random CSRF nonce, stores it in sessionStorage, and hands the browser to LinkedIn.
LinkedIn OpenID Connect	The identity provider. Authenticates the visitor and returns a single-use authorization code under the openid, profile and email scopes.
API Gateway (HTTP API)	The managed front door. A single GET /callback route with Lambda proxy integration and an auto-deploying stage. Terminates TLS and shields the function from the raw internet.
Lambda — recruiter-verifier-backend	The decision engine. Exchanges the code for an access token, gathers every claim LinkedIn will release, runs the five-test policy, mints or withholds the approval token, and emits a structured audit record.

Component	Responsibility
Signed hand-off token	HMAC-SHA256 over a compact payload with a 15-minute TTL, a unique jti replay identifier and constant-time verification.
Outcome pages	verified.html greets an approved visitor and forwards to the live calendar; not-verified.html reads a reason code and renders a specific explanation.
CloudWatch	The system of record. Every request logs the policy in force, each test's state, whether data was actually available, and the final verdict.

3.2 The Request Lifecycle

- **Initiation.** The browser generates 16 bytes of cryptographic randomness as a CSRF nonce, persists it, and redirects to LinkedIn with the client id, exact registered redirect URI, scopes and nonce.
- **Authentication.** LinkedIn authenticates the visitor, obtains consent, and redirects back to the registered callback with a single-use authorization code.
- **Reception.** API Gateway matches GET /callback and invokes the function through proxy integration, passing the query string intact.
- **Exchange.** The function posts code, client credentials and redirect URI to the token endpoint. A reused or expired code produces a retry message, not a verification failure.
- **Claim gathering.** The function requests the OIDC userinfo document and, in parallel, attempts the privileged verification-report and headline endpoints, tolerating denial.
- **Adjudication.** Five tests each return PASS, FAIL or UNAVAILABLE; the policy engine applies a mandatory set plus a minimum passing count.
- **Hand-off.** Approved: a signed token is minted and the browser redirected onward. Rejected: redirected with a specific reason code.
- **Audit.** A structured JSON record of the whole decision is written to CloudWatch.

4. Technology Stack

The stack was chosen for operational minimalism: every component is managed, scales to zero, and bills only for what is consumed.

Layer	Technology	Role in the System
Compute	AWS Lambda · Node.js 24.x	Stateless verification engine. 512 MB, 30 s timeout, no VPC attachment.
API surface	API Gateway — HTTP API	Managed TLS front door; GET /callback with Lambda proxy integration.
Identity	LinkedIn OpenID Connect	Federated authentication; authorization-code flow with openid, profile, email.

Layer	Technology	Role in the System
Storage	Amazon S3	Private origin bucket holding the site, outcome pages and media.
Delivery	CloudFront + OAC	Global edge distribution over a private origin; query-string forwarding.
Observability	Amazon CloudWatch Logs	Structured decision records, cold-start diagnostics, duration and memory reports.
Access control	AWS IAM	Execution role scoped to log writes; no standing data-plane permissions.
Cryptography	HMAC-SHA256 (node:crypto)	Signed, expiring tokens with constant-time verification and replay identifiers.
Front end	HTML5 · CSS3 · vanilla JS	Hand-built responsive portfolio; no framework, no build step, no dependencies.
Scheduling	Calendly	The protected resource — the calendar verification exists to defend.
Applied AI	AI avatar video resume	Synthetic presenter on the author's likeness and voice, embedded in the portfolio.

5. The Verification Engine — Technical Deep Dive

This section is the intellectual core of the project. Everything else is competent cloud assembly; this is where the engineering judgement lives.

5.1 The Failure That Shaped the Design

The original implementation ran three checks and required all three to pass: a LinkedIn verification badge, a recruiter keyword in the headline, and a corporate email domain. It was logically clean and it rejected **every single visitor**.

The cause was a category error. Two of the three checks depend on privileged endpoints. The verification-report endpoint returns HTTP 403 to applications without elevated access, and the headline field requires a scope standard-tier applications are not issued. The original code read those failures as 'no badge' and 'no keyword'. In reality they meant 'LinkedIn declined to tell you.' The system was punishing visitors for its own lack of permission.

Absence of evidence was being scored as evidence of absence — producing a gate with a one hundred percent false-rejection rate.

5.2 The Three-State Model

The rewrite replaced binary pass/fail with a three-valued result. The distinction between the second and third states is the entire fix:

State	Meaning	Effect on the verdict
PASS	The provider returned data and the visitor satisfies the rule.	Counts toward the passing total.
FAIL	The provider returned data and the visitor violates the rule.	A genuine failure; can veto if mandatory.
UNAVAILABLE	The provider returned nothing. Not evidence either way.	Never counted against the visitor.

Each test also reports a `dataAvailable` flag, so the operator can see which sources are actually answering in production — and therefore which rules can safely be made mandatory later.

5.3 The Five Tests

Test	Source	Passes when	Readable
openid	OIDC userinfo	A valid account identifier (sub) is returned	Yes
profile	OIDC userinfo	The account has both a display name and a photo	Yes
badge	verificationReport	Categories include WORKPLACE or IDENTITY	No — 403

Test	Source	Passes when	Readable
headline	v2/me projection	Headline contains a recruiter keyword	No — scope
domain	OIDC userinfo	Email is corporate — not consumer, not disposable	Yes

The domain test screens against a curated deny-list of more than fifty consumer, international and disposable providers, and rejects malformed domains. It is the one discretionary test whose data is reliably available today, making it the natural next candidate for promotion to mandatory status.

5.4 The Policy Engine

Admission rules are configuration, not control flow. Two declarations govern the gate:

```
const MIN_PASSING_CHECKS = 2;
const ALWAYS_REQUIRED = ["openid", "profile"];
```

ALWAYS_REQUIRED is a veto list: every named test must actively PASS. MIN_PASSING_CHECKS is a floor on total passes. Because the two mandatory tests also satisfy the count, they are currently both necessary and sufficient — a deliberate, documented choice while the privileged sources remain unreadable.

The effect is precise and intentional: an authenticated account with a real name and photo is admitted. A recruiter on a consumer email address is still admitted — their domain test fails, the failure is recorded and carried in their token, but it does not block them. The rule is enforced-and-logged rather than enforced-and-blocking, so policy can be tightened later on real data instead of guesswork.

Goal	Change
Make corporate email mandatory	ALWAYS_REQUIRED = ["openid", "profile", "domain"]
Require one criterion beyond the pair	MIN_PASSING_CHECKS = 3
Loosen to any two of five	ALWAYS_REQUIRED = []

The documented trap: never promote a test to mandatory while its `dataAvailable` flag reads false in CloudWatch. That single sentence in the operations notes encodes the exact mistake that broke the first deployment — captured so it cannot be repeated.

5.5 Exhaustive Verification

A five-test policy over three states is a finite state space, so it was not sampled — it was **enumerated**. All 162 valid combinations were generated and each evaluated against the specification 'access is granted if and only if openid and profile both pass.'

162 of 162 combinations correct. The policy engine is not believed to be right; it is demonstrated to be right, across its entire input space.

5.6 The Signed Hand-off Token

Approval must survive a redirect into a static page, so it cannot rely on server session state. The function mints a compact signed credential:

- **Payload.** Account subject, email, display name, tests passed, tests skipped as unavailable, issued-at and expiry timestamps, and a UUID replay identifier.
- **Signature.** HMAC-SHA256 over the base64url-encoded payload — a deliberately minimal JWT-shaped structure without inheriting JWT's algorithm-confusion pitfalls.
- **Expiry.** Fifteen minutes. Long enough to book a meeting, short enough that a leaked link is worthless almost immediately.
- **Constant-time verification.** Signatures compared with a timing-safe primitive after an explicit length check, closing a timing side-channel.
- **Replay identity.** Every token carries a unique jti, giving a ready hook for single-use enforcement.

5.7 Honest Assessment of the Hand-off

The security review recorded in the project's own operations notes is candid about the remaining gap. The token proves the function approved that person, but the success page is a static document — its token check is cosmetic, because a static page cannot hold a signing key. The booking URL remains publicly reachable, so anyone who already knows it can book directly.

The gate therefore raises the cost of abuse substantially without being an absolute boundary, and the documented remediation is specific: mint single-use booking links through the scheduling provider's API inside the Lambda. **Knowing precisely where your security boundary actually sits is itself a security control.**

5.8 Failure Taxonomy

Rejection is never a dead end. Twelve reason codes are carried to the failure page, separating genuine policy failures from mechanical ones like a reused single-use code, which is a retry condition rather than a judgement about the visitor.

Code	Meaning	Code	Meaning
no_identity	No valid account id	no_email	No verified email returned
incomplete_profile	Missing name or photo	insufficient_data	Too few readable criteria
no_badge	Readable, none present	code_expired	Sign-in link reused/expired
no_keyword	Readable, no match	access_denied	Visitor cancelled at LinkedIn
consumer_domain	Consumer/disposable	bad_state	Callback hit without a code
token_error	Identity handshake failed	server_error	Unhandled exception

6. Implementation Walkthrough

What follows is the build in the order it happened — provisioning, integration, failure, diagnosis, correction and live validation. Every screenshot is an authentic capture taken during the work, and each is accompanied by a short explanation of what is happening and why the step exists. In keeping with sound security practice, all account numbers, resource and deployment identifiers, endpoints, bucket names, distribution identifiers, browser address bars, client secrets, signing keys and OAuth tokens visible in any capture have been permanently redacted.

6.1 Build Artifacts

The deployment package separates concerns cleanly and treats infrastructure configuration as a first-class, version-controlled artifact.

The deployment package

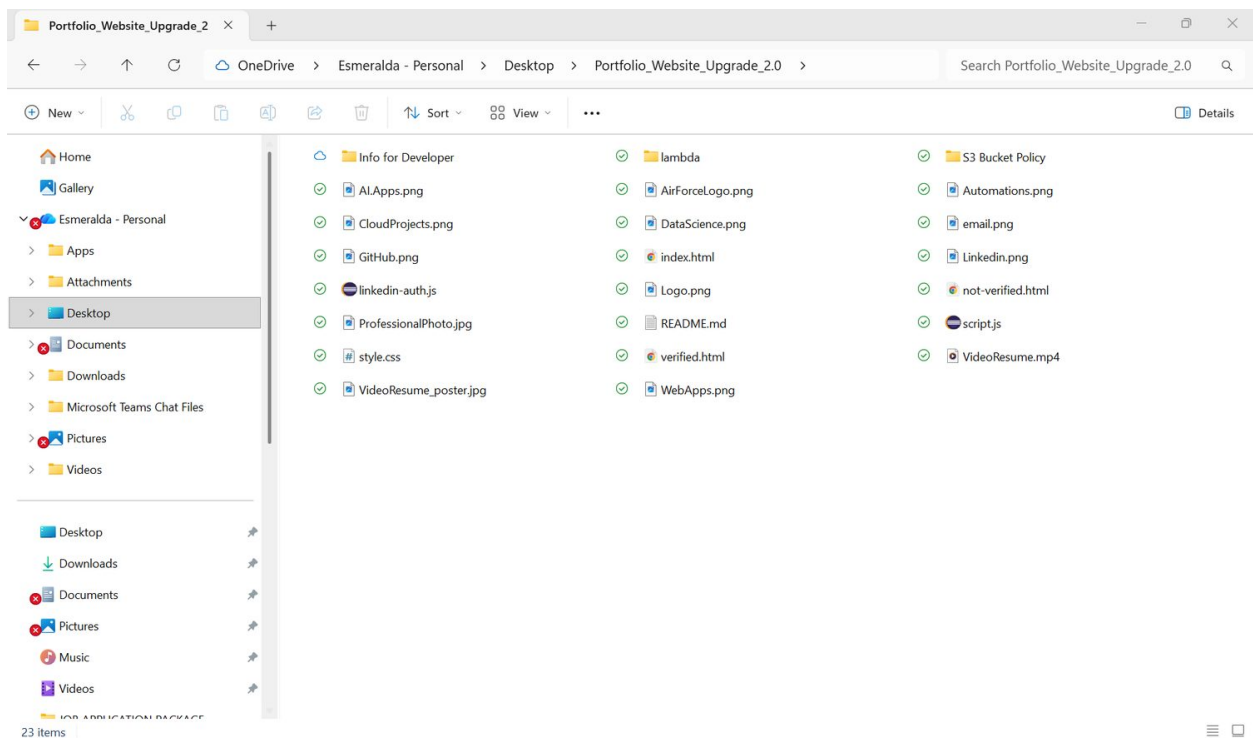


Figure 2. Local build artifacts staged for deployment.

The complete 2.0 deployment package on disk — front-end assets and the two outcome pages at the root, the Lambda backend isolated in its own folder, and the S3 bucket policy stored as a JSON file. Separating the backend from the static assets lets each deploy independently, and keeping the bucket policy as a versioned file means the security configuration is reviewable code rather than undocumented console clicks.

6.2 The Protected Resource — Scheduling Layer

Design started at the end of the chain. Before building a gate it is worth being precise about exactly what sits behind it.

Defining the availability windows

The screenshot shows the Calendly interface for setting availability. The main section is titled "When are you available to meet with people?" with a subtext "You'll only be booked during these times (you can change these times and add other schedules later)". Below this is a "Weekly hours" section with a table of availability slots for each day of the week. A "Time zone" dropdown is set to "Pacific Time - US & Canada (11:58pm)". A modal titled "Select a Date & Time" is open, showing a calendar for August and a list of available times for Wednesday, August 16: 10:00am, 10:30am, 11:00am, and 11:30am. The 11:00am slot is selected, and a "Confirm" button is visible.

Day	Start Time	End Time
S	8:00am	5:00pm
M	5:00pm	9:00pm
T	5:00pm	9:00pm
W	5:00pm	9:00pm
T	5:00am	9:00pm
F	4:00pm	9:00pm
S	8:00am	5:00pm

Time zone: Pacific Time - US & Canada (11:58pm)

Select a Date & Time

WEDNESDAY, AUGUST 16

- 10:00am
- 10:30am
- 11:00am
- 11:30am

Confirm

Figure 3. Defining weekly availability windows in the scheduling layer.

Setting the specific hours during which meetings may be booked. Design started at the end of the chain, because it is worth being precise about what the gate protects. The calendar is deliberately bounded before it is ever exposed — verification defends a finite, shaped surface rather than an open-ended invitation.

The integration catalogue

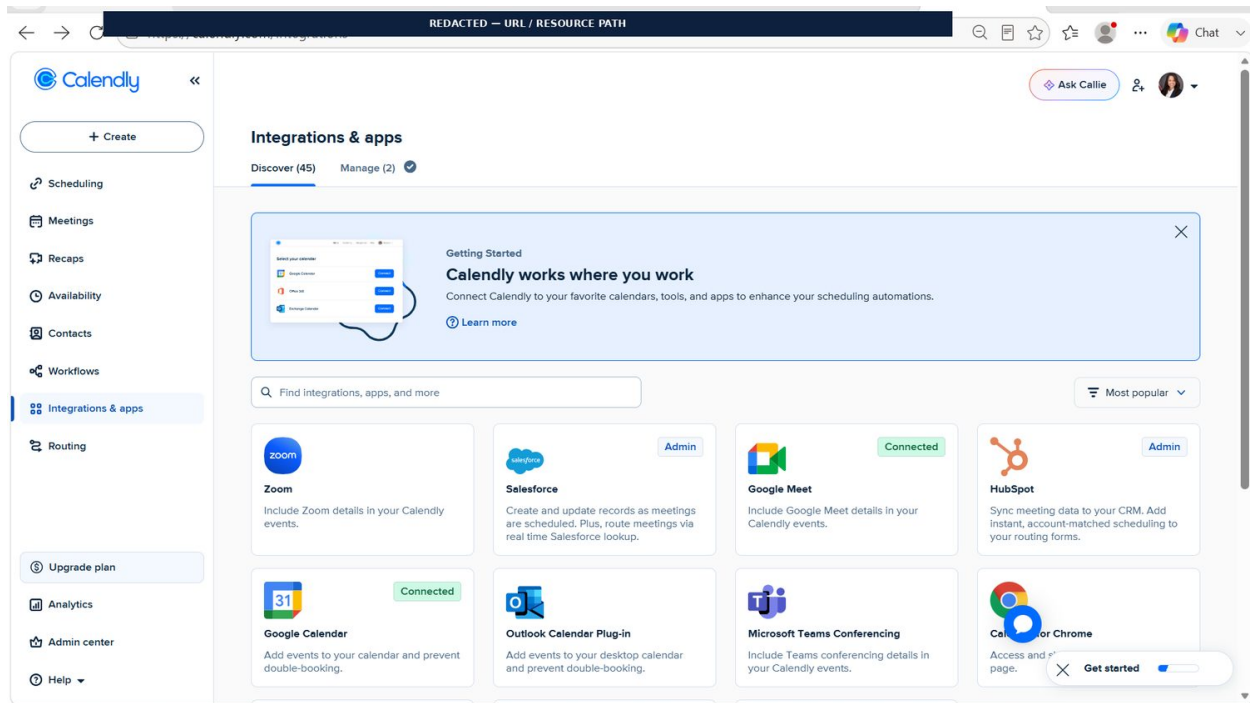


Figure 4. Browsing the scheduling platform's integration catalogue.

Locating the calendar integration that will connect scheduling to a real, live calendar. A booking system that does not reflect actual commitments will cheerfully double-book. Verification is worthless if the meeting it grants cannot actually be honoured.

Connecting the live calendar

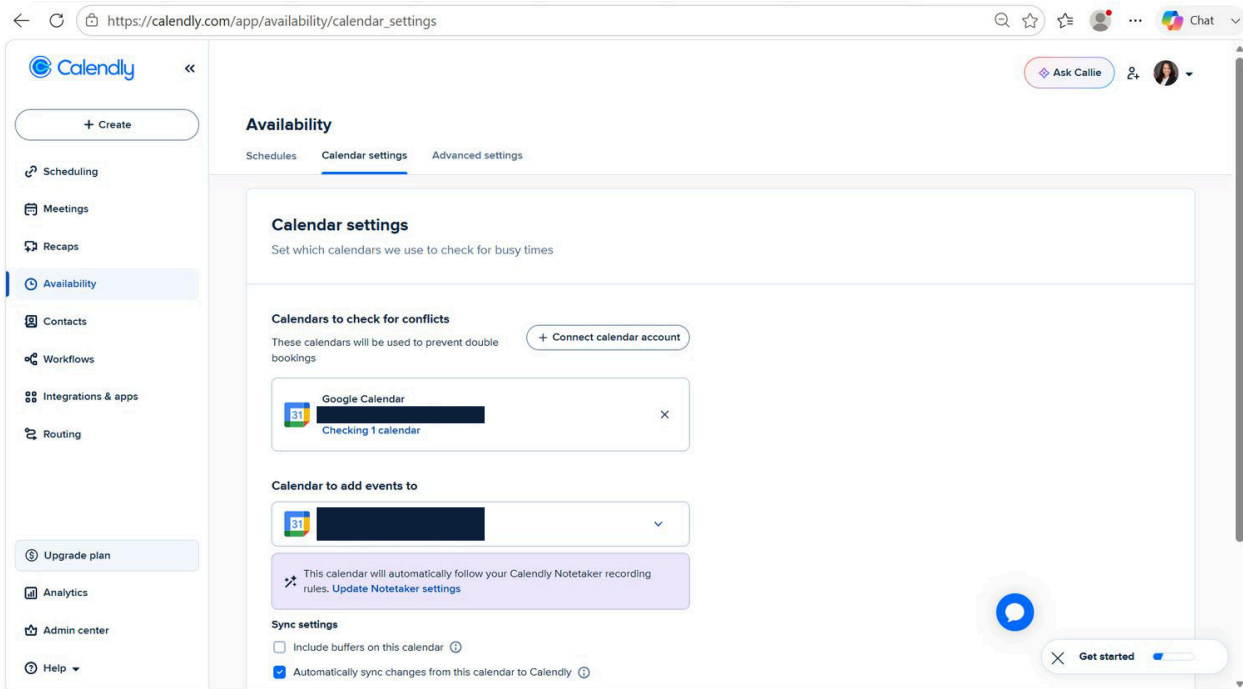


Figure 5. Google Calendar connected to the scheduling layer.

The calendar connection is confirmed and active. Bidirectional synchronisation means existing obligations automatically block availability, so an approved visitor can only ever select a slot that is genuinely free.

The published booking event

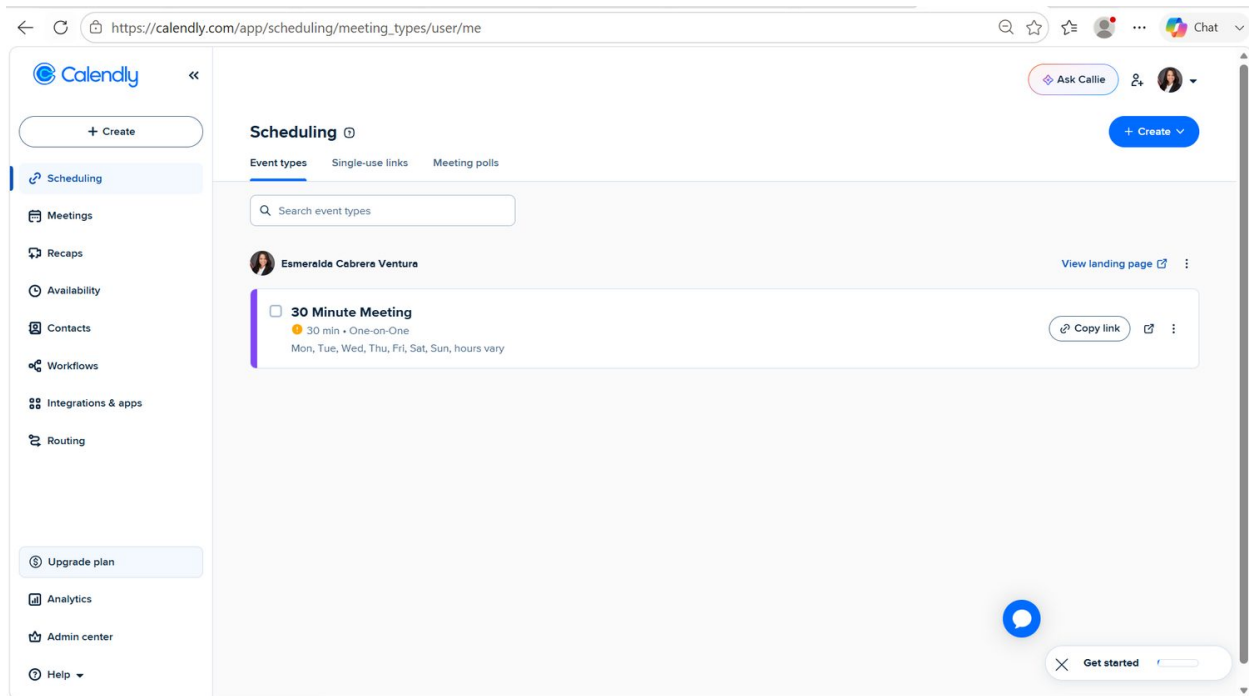


Figure 6. The published scheduling profile — a 30-minute one-on-one event type.

The live, publicly addressable booking event that the system will hand approved visitors to. This URL is the protected resource. Everything built in the remaining fifty screenshots exists for one purpose: to control who reaches this page.

6.3 Identity Provider Registration

The gate delegates authentication to LinkedIn, which requires registering a first-class application and taking custody of its credentials — the project's first real security boundary.

Registering the OAuth application

The screenshot shows the LinkedIn Developer Portal interface for a specific application named "Recruiter Verifier App". The top navigation bar includes the LinkedIn logo, "DEVELOPERS", and links to "Products", "Docs and tools", "Resources", and "My apps". The application details show it was created on Jul 17, 2026, and is a "Standalone app". The "Settings" tab is active, displaying fields for "App name" (Recruiter Verifier App), "Privacy policy URL" (https://www.esmeraldaspace.com), and "App logo". A "Verify" button is present next to the "Default Company Page for Individual Developer" section. A "Delete app" button is located at the top right of the settings area. A notification banner at the top of the settings section prompts the user to "Update your business email for receiving critical platform updates. Update email". The right sidebar contains a "Managing settings for your app" section with instructions on how to adjust application settings, including app logo and privacy policy URL, and a "Widgets" section with instructions on how to add and validate domains for widgets. A "Leave Feedback" button is visible on the right edge of the page.

Figure 7. Registering the verification application in the LinkedIn Developer Portal.

Creating the OAuth client that will request identity assertions on the site's behalf. OAuth 2.0 requires a pre-registered client — this is what establishes the application's identity with the provider and causes credentials to be issued. Without it there is no federated identity and no gate.

The client credentials

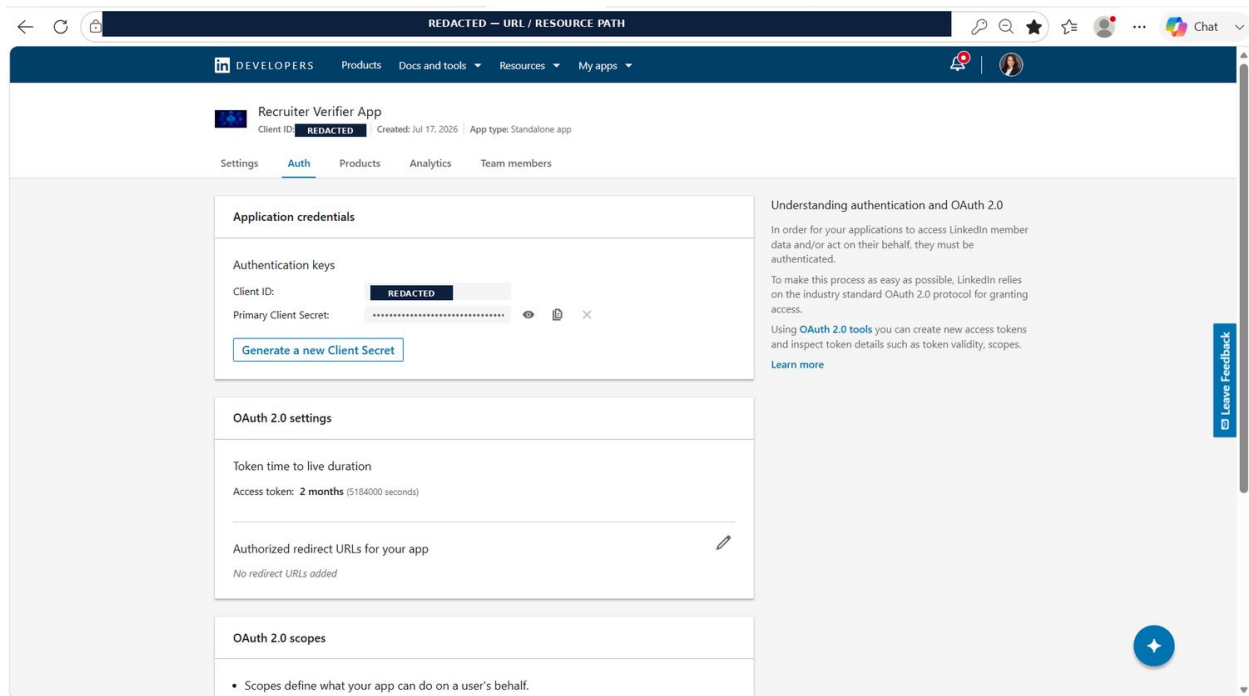


Figure 8. Application authentication settings — client identifier and primary client secret. (Credentials redacted.)

The credential pair that authenticates the backend to LinkedIn during the token exchange. The client secret is the single most sensitive value in the project. It must live only server-side inside the Lambda and never reach a browser. It is redacted here and has been rotated.

6.4 The Managed Front Door — API Gateway

LinkedIn will only redirect to a pre-registered HTTPS endpoint, and exposing a function directly to the internet is poor practice. A managed HTTP API solves both problems with a single tightly-scoped route.

The AWS console

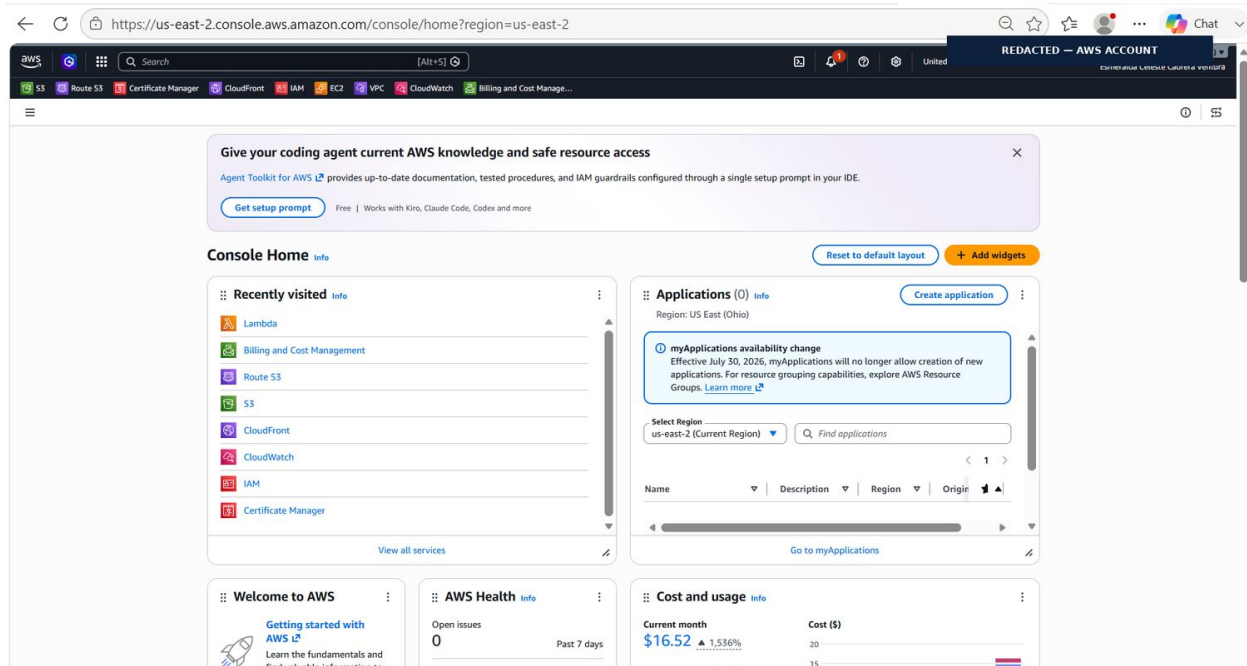


Figure 9. The AWS Management Console. (Account identifier redacted.)

The account and region context in which every cloud resource will be created. Region selection is not cosmetic. The API Gateway endpoint carries its region in the hostname, and that hostname is registered with LinkedIn as the redirect URI — so choosing a region here permanently shapes a value the identity provider must match exactly.

The API Gateway console

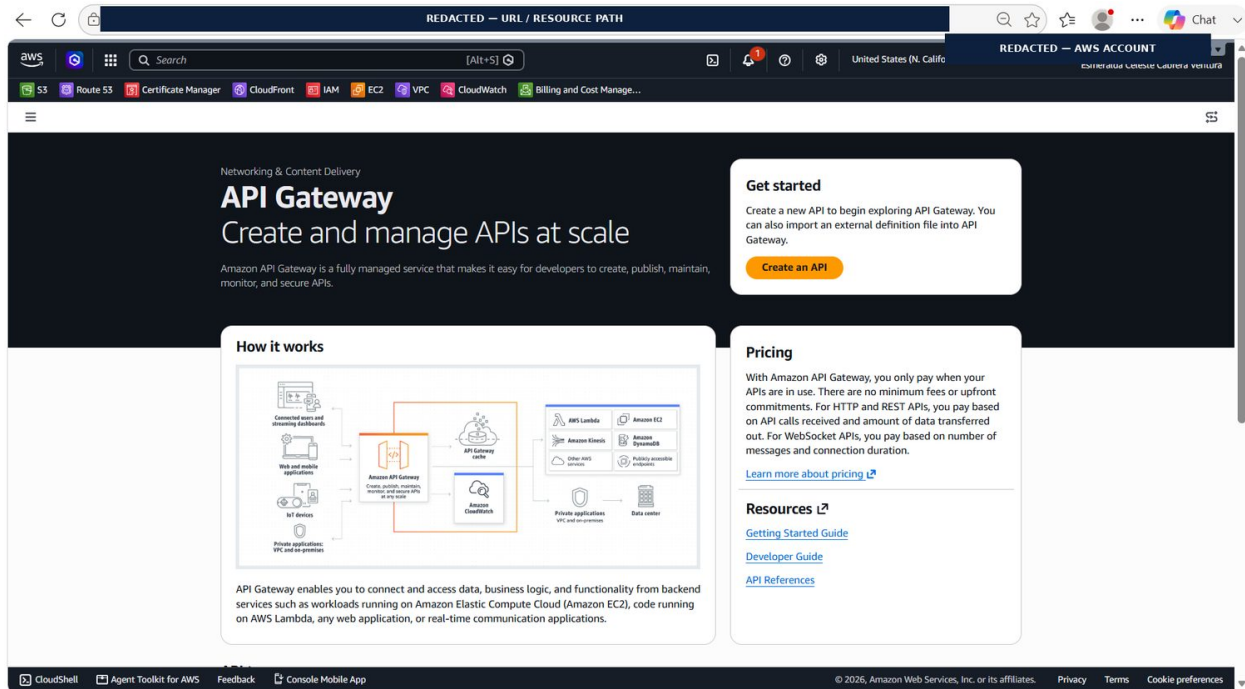


Figure 10. The API Gateway service console — entry point for building the HTTP front door.

Opening the service that will expose the verification function to the internet. Two constraints force this component to exist. LinkedIn will only redirect to a pre-registered HTTPS endpoint, and exposing a Lambda function directly to the public internet is poor practice. A managed gateway solves both.

Choosing the HTTP API

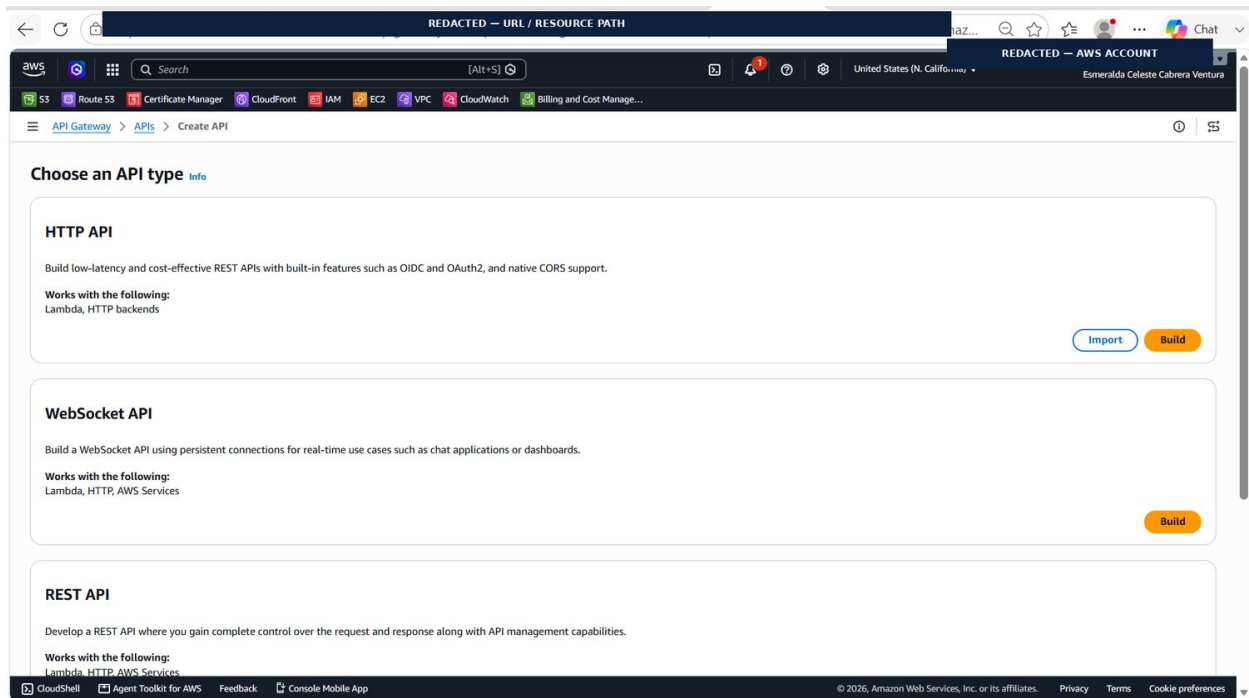


Figure 11. Selecting HTTP API rather than REST API.

Choosing between API Gateway's two product families. The HTTP API offers lower latency, materially lower cost, and native Lambda proxy integration. The REST API's extra capabilities — request validators, usage plans, API keys — are unnecessary for a single OAuth callback route, and unused features are unnecessary surface.

Binding the Lambda integration

The screenshot shows the AWS Management Console interface for creating a new HTTP API. The breadcrumb navigation indicates the path: API Gateway > APIs > Create API. The left sidebar shows the progress of the setup steps: Step 1: Configure API (selected), Step 2: optional: Configure routes, Step 3: optional: Define stages, and Step 4: Review and create. The main content area is titled 'Configure API' and contains the following sections:

- API details**
 - API name**: A text input field containing 'recruiter-verifier-gateway'.
 - IP address type**: A radio button selection between 'IPv4' (selected) and 'Dualstack'.
- Integrations (1)**
 - A dropdown menu showing 'Lambda'.
 - A table of integrations with columns for 'AWS Region' (us-west-1), 'Lambda function' (REDACTED), and 'Version' (2.0). A 'Remove' button is next to the integration.
 - An 'Add integration' button at the bottom.

At the bottom right of the configuration area are buttons for 'Cancel', 'Review and create', and 'Next'.

Figure 12. Configuring the API and binding the Lambda proxy integration. (Function ARN redacted.)

Naming the API and attaching the verification function as its backing integration. This is the binding that makes an inbound HTTPS request become a function invocation. The ARN identifies exactly which function in which region receives the callback.

Defining the /callback route

The screenshot shows the AWS API Gateway console interface. On the left, a sidebar lists the steps: Step 1: Configure API, Step 2 - optional: Define stages, Step 3 - optional: Configure routes (highlighted), Step 4: Review and create. The main panel is titled 'Configure routes - optional'. It contains a section 'Configure routes' with an 'Info' icon. Below this, there's a text block explaining that API Gateway uses routes to expose integrations and that routes consist of an HTTP method and a resource path. The form has three fields: 'Method' (a dropdown menu with 'GET' selected), 'Resource path' (a text input field containing '/callback'), and 'Integration target' (a dropdown menu with 'recruiter-verifier-backend' selected). There is a 'Remove' button next to the integration target. Below these fields is an 'Add route' button. At the bottom right of the main panel, there are four buttons: 'Cancel', 'Review and create', 'Previous', and 'Next'.

Figure 13. Defining the single route: GET /callback.

Creating one route, one method, mapped to the verification function. This is the entire public attack surface of the system — a single GET path. The path must also match the redirect URI registered at LinkedIn character for character.

Configuring the deployment stage

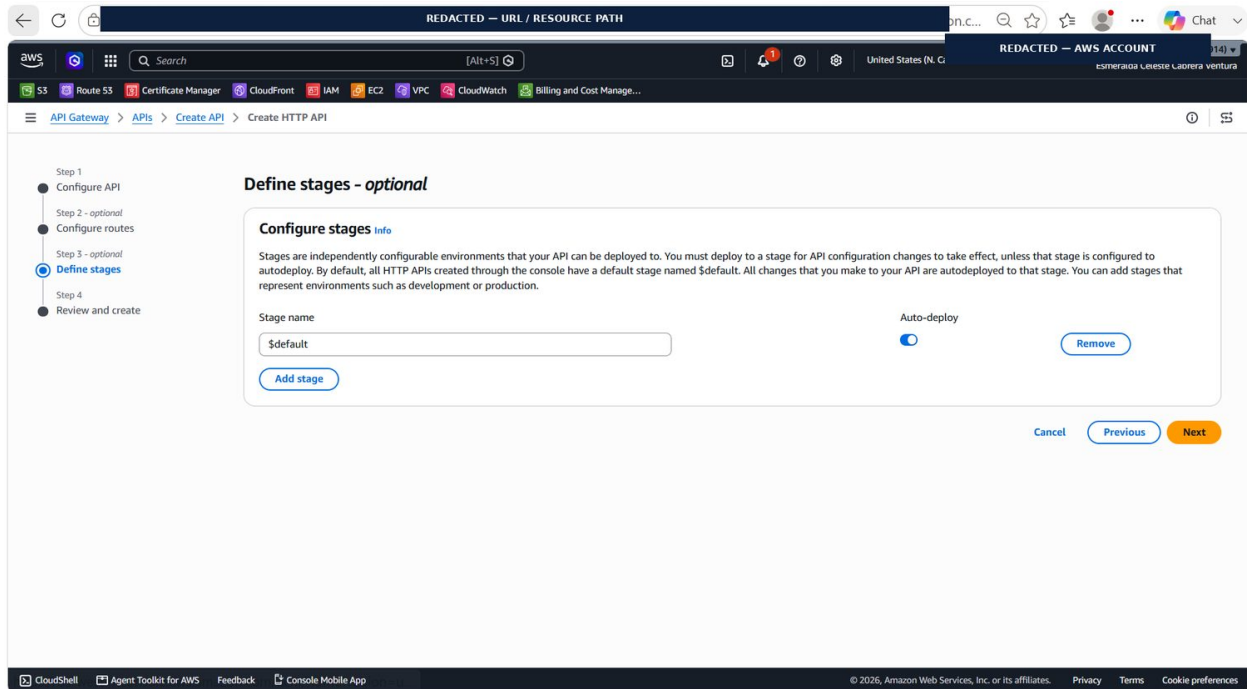


Figure 14. Defining the deployment stage.

Configuring the stage that publishes the API and enabling automatic deployment. The stage name becomes part of the invoke URL. Using the default stage keeps the URL clean, which keeps the registered redirect URI simple — a named stage would insert an extra path segment and silently invalidate every registered URI.

The API goes live

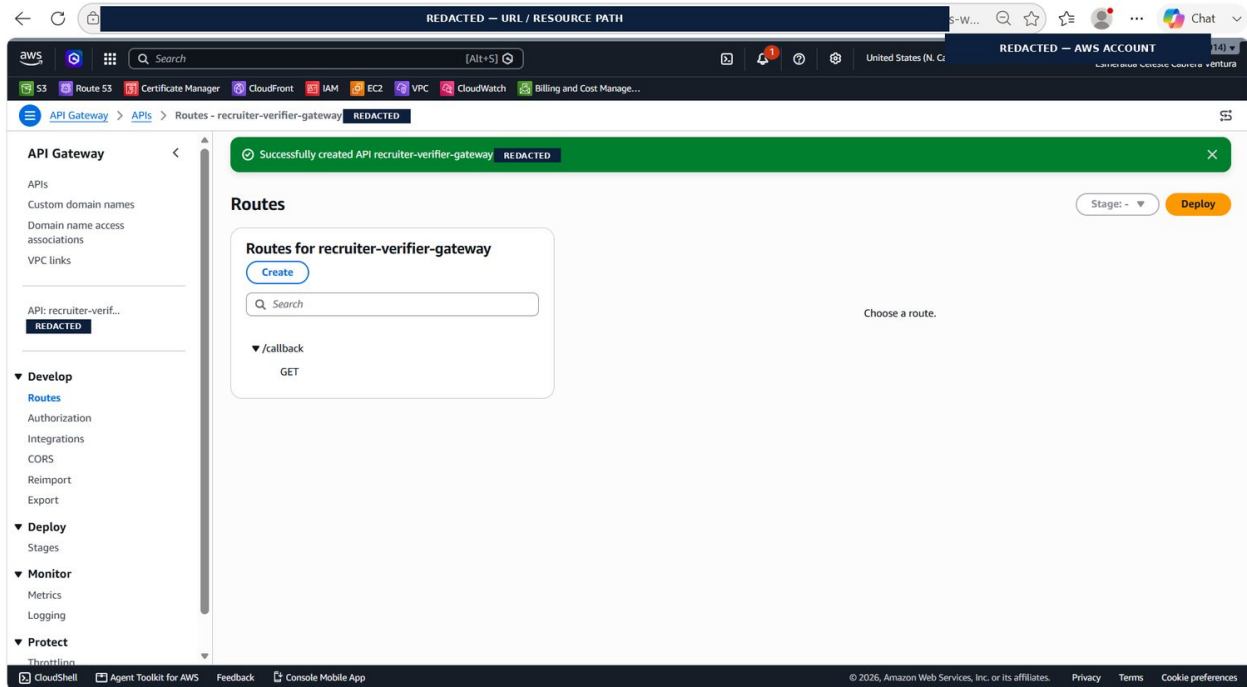


Figure 15. The API created successfully with the callback route live. (Identifiers redacted.)

Confirmation that the API exists and the route is deployed. The generated API identifier becomes part of the public endpoint hostname, which is why it is redacted throughout this report — it is the address of the only internet-facing component in the architecture.

Capturing the invoke URL

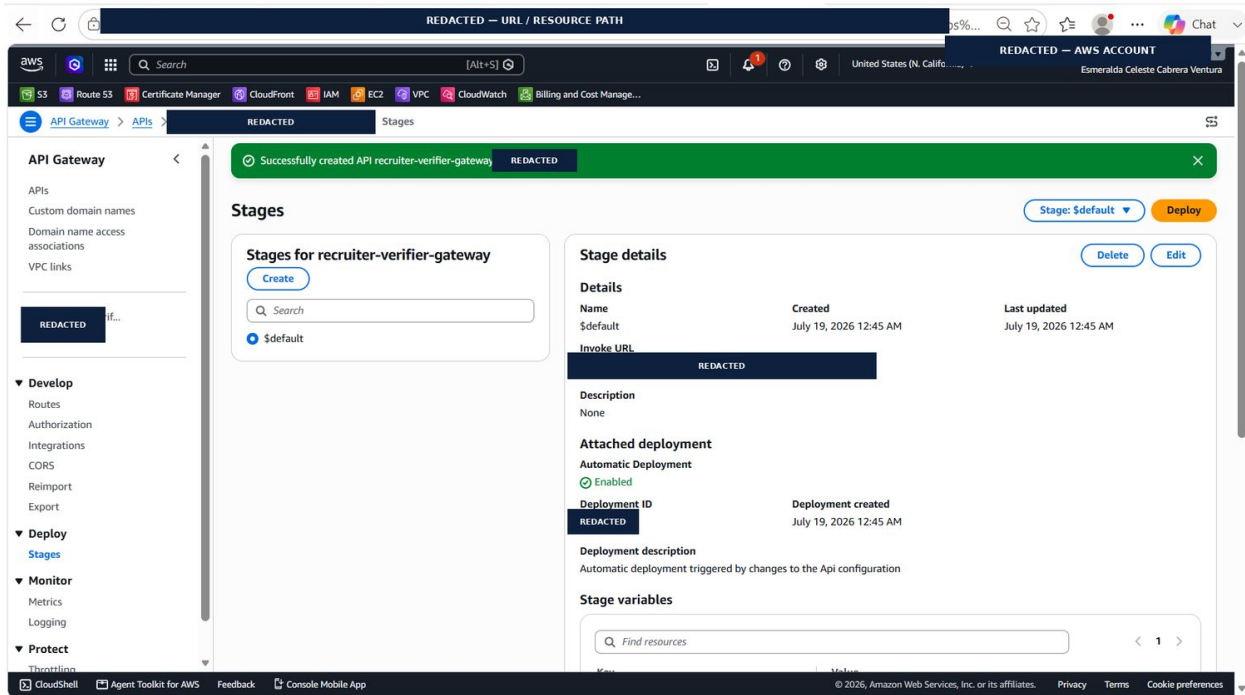


Figure 16. Stage details and the live invoke URL. (Endpoint and deployment identifiers redacted.)

Retrieving the exact HTTPS address that the identity provider must redirect back to. This string has to be reproduced identically in three separate places — the LinkedIn app registration, the front-end configuration, and the function's own constant. A single character of drift produces a redirect URI mismatch and the entire flow fails before it starts.

6.5 Binding the Redirect URI

Redirect-URI registration is an anti-phishing control. The invoke URL, the front-end configuration and the function's own constant must agree exactly.

Registering the redirect URI

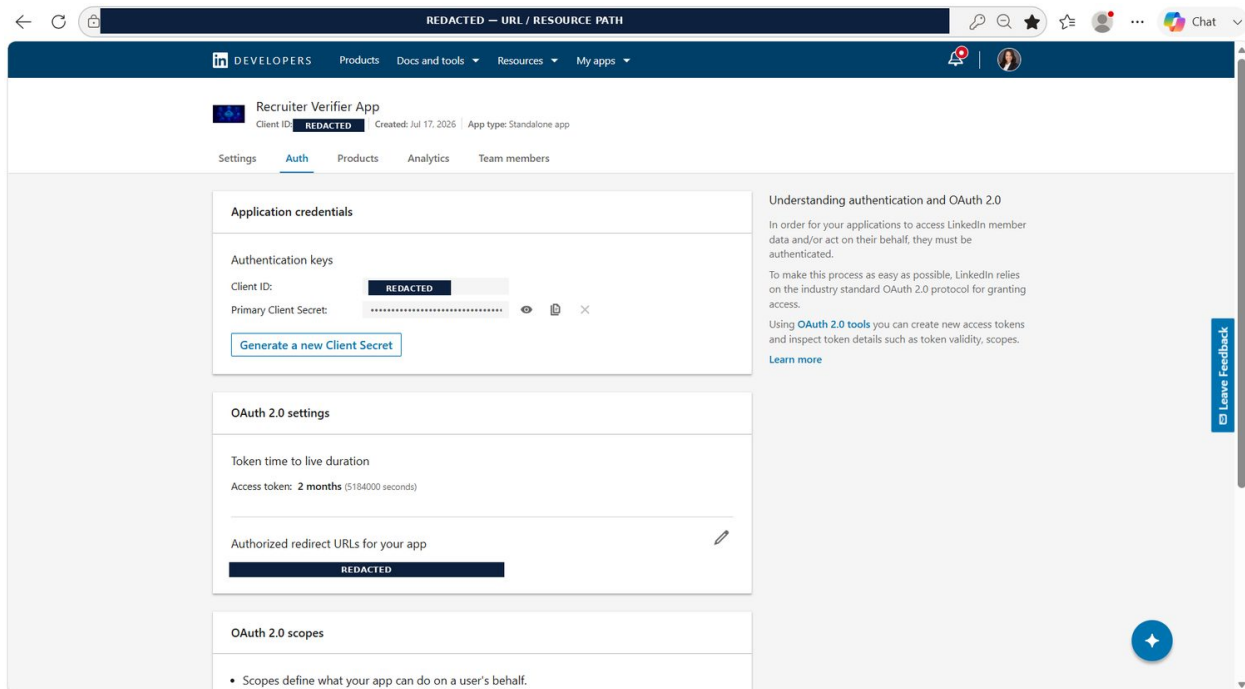


Figure 17. Registering the live invoke URL as an authorized redirect URL. (Endpoint redacted.)

Declaring at the identity provider exactly where authorization codes may be sent. This is an anti-phishing control. Because the provider will only release a code to a pre-declared location, an attacker who steals the client ID still cannot redirect identity assertions to a server they control.

Confirming the OIDC grant

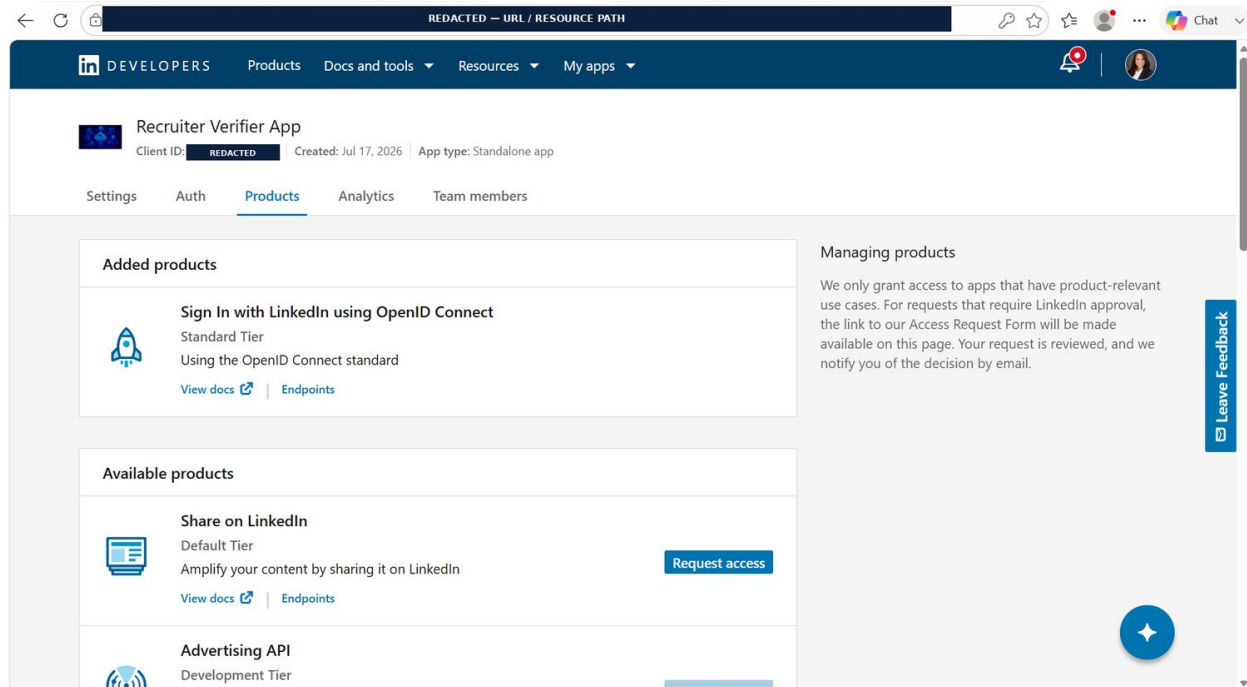


Figure 18. Confirming Sign In with LinkedIn using OpenID Connect is an active product.

Verifying the grant that makes the openid, profile and email scopes available to this application. This screen also explains the central technical constraint of the whole project. Standard-tier access grants these three scopes and no more — which is precisely why the verification-badge and headline data later return 403, and why the policy engine had to be redesigned to tolerate missing data.

6.6 The Verification Function

With the front door and the identity provider bound together, the decision engine itself was created, configured and deployed.

The Lambda console

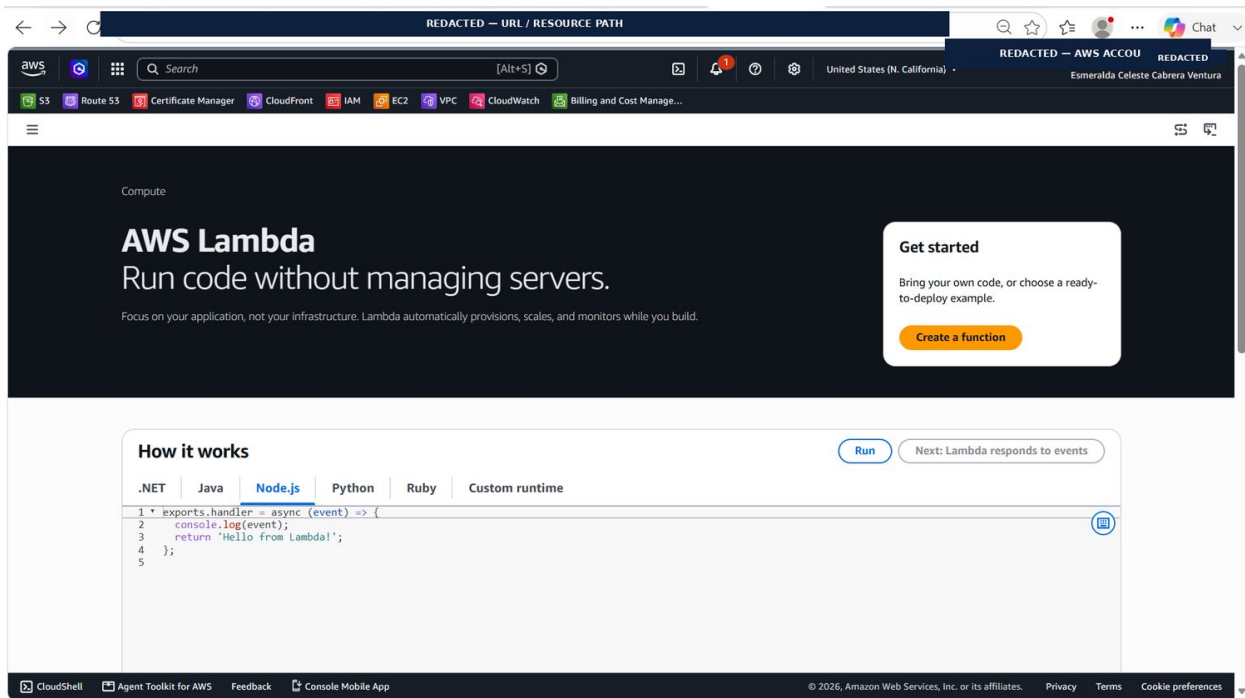


Figure 19. The Lambda console.

Opening the compute service that will host the decision engine. Serverless was chosen deliberately. Verification runs for a few hundred milliseconds a handful of times a day; provisioning a continuously running server for that workload would mean paying for idle capacity and owning a patching obligation for nothing.

Creating the verification function

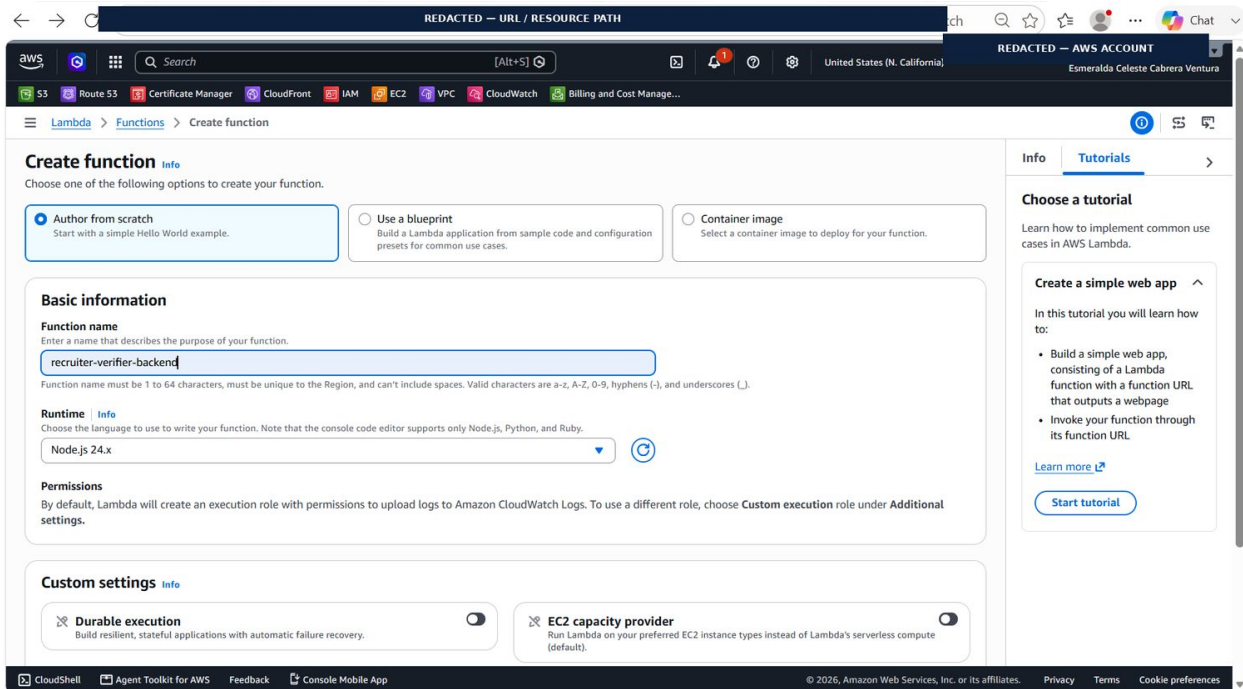


Figure 20. Creating the verification function on the Node.js 24.x runtime.

Authoring the function from scratch on a current Node runtime. A modern runtime provides native fetch and ES module support, which lets the entire OAuth orchestration be written with no third-party dependencies at all. Zero dependencies means zero supply-chain surface and nothing to keep patched.

The function comes online

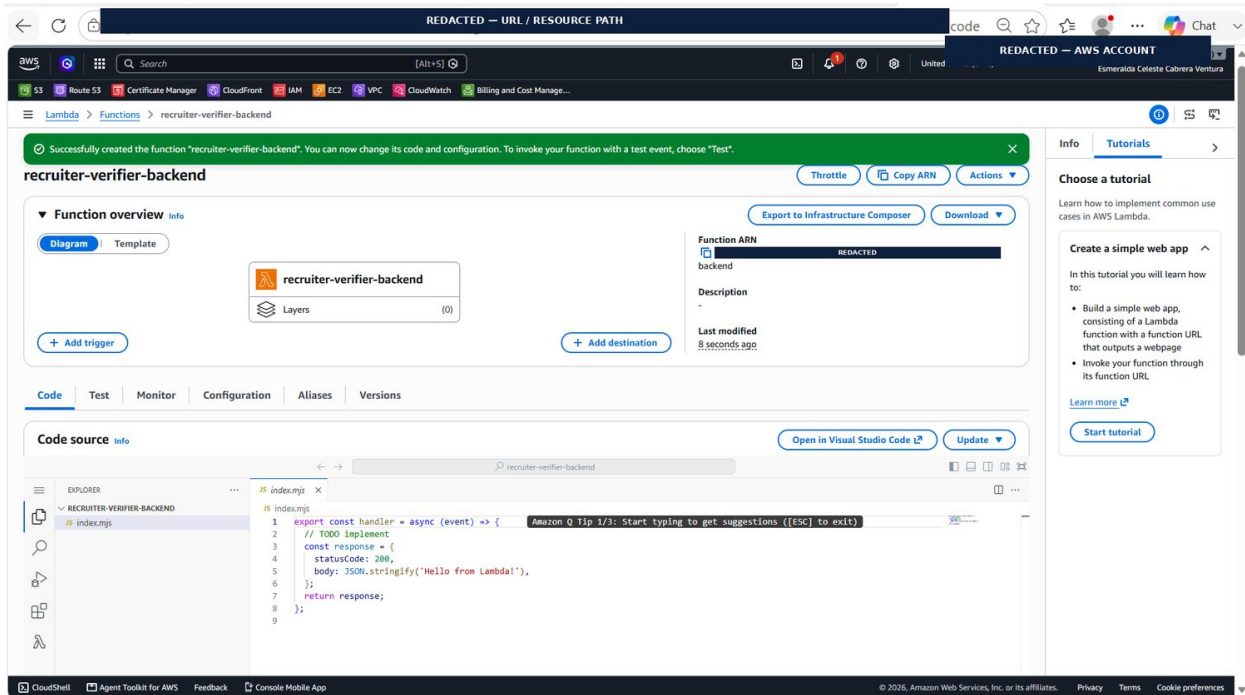


Figure 21. Function created successfully. (ARN and account identifier redacted.)

The function now exists and is addressable by its ARN. This ARN is what the API Gateway integration created earlier points at, completing the path from public HTTPS endpoint to executable code.

Deploying the verification engine

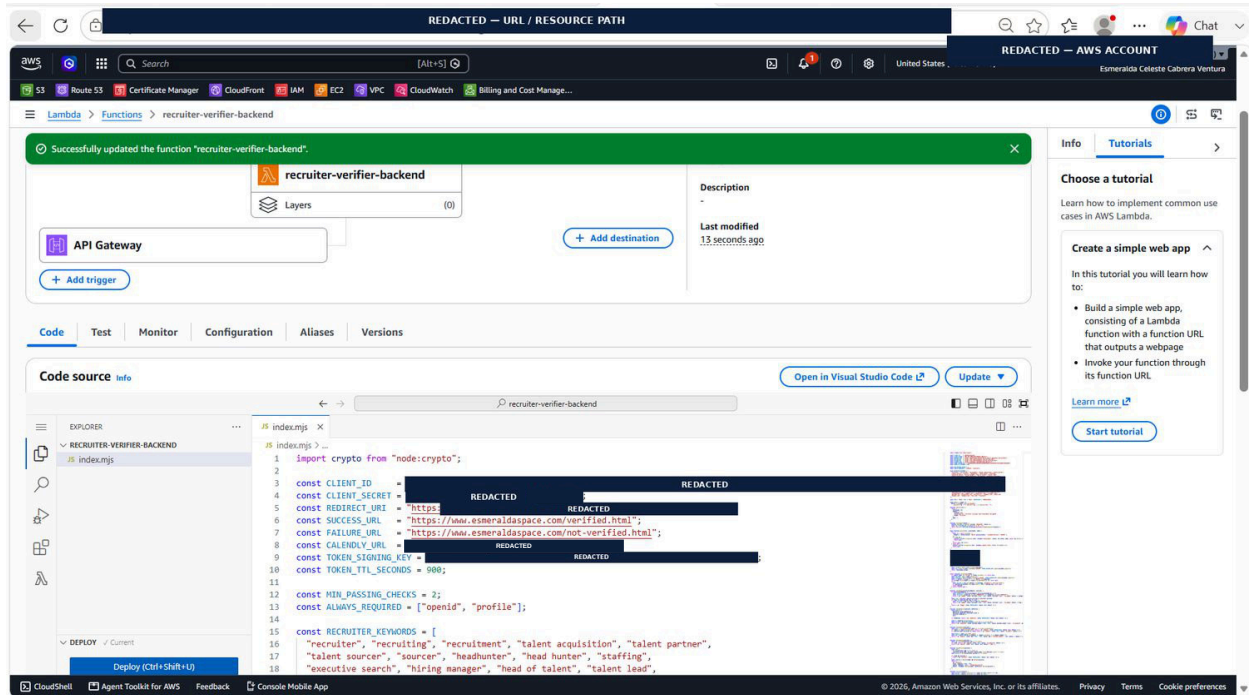


Figure 22. The verification engine deployed — configuration constants and policy declarations. (Credentials and signing key redacted.)

The real source of the rebuilt engine, showing the configuration block, the two policy declarations that govern admission, and the recruiter keyword list. This is policy-as-configuration in practice — the admission rules can be tightened by editing two lines rather than rewriting logic. It also shows why storing secrets as module constants is flagged in this report as the next hardening step.

6.7 First Live Test — Total Failure

The first end-to-end test failed outright, and it failed in the least helpful way possible.

The first live test

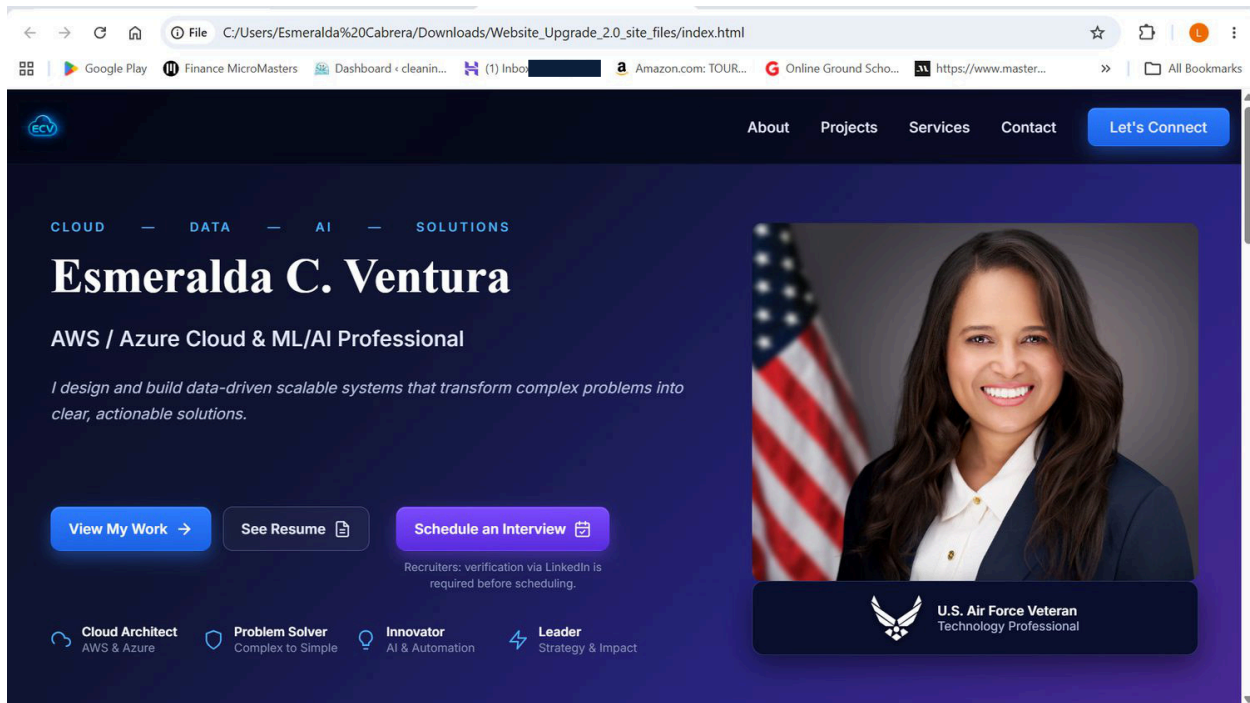


Figure 23. Exercising the Schedule an Interview control on the live site.

The first true end-to-end test, driven through real infrastructure exactly as a recruiter would experience it. Every component had been individually configured and looked correct. Integration testing is where that assumption gets destroyed.

The failure surfaces

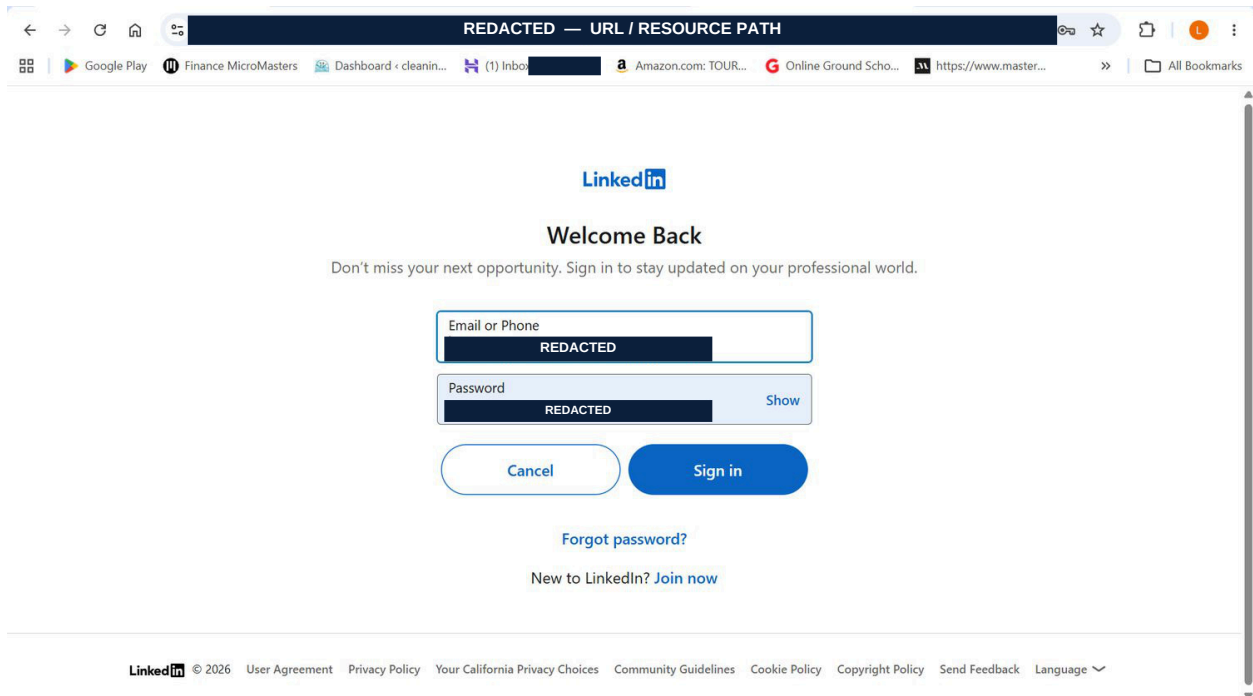


Figure 24. The first attempt fails — a generic *Internal Server Error*.

The gateway returns an opaque HTTP 500 instead of redirecting the visitor. This is the least informative failure a system can produce. It means the function either crashed or never returned at all, leaving the gateway with no valid response to relay. Critically, it is impossible to diagnose from the browser — nothing here indicates the cause.

Inspecting the failing callback

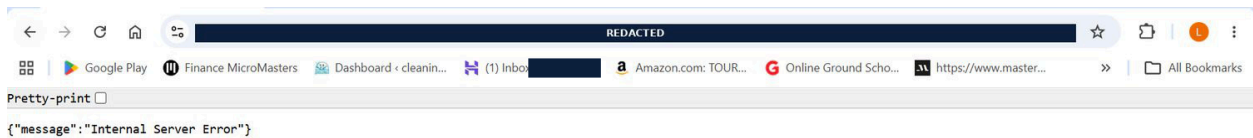


Figure 25. The failing callback request. (Endpoint and live authorization code redacted.)

Examining the actual callback URL, which carries a freshly issued authorization code. This proves the identity provider did its job correctly — a valid code was issued and delivered. The failure is therefore entirely downstream, which narrows the investigation to the gateway and the function.

6.8 Root-Cause Analysis

This is, professionally speaking, the most valuable sequence in the report. A generic 500 is not a diagnosis — it is an invitation to guess. Instead the failure was pursued through three progressively more authoritative layers of evidence.

First layer — the rendered page source

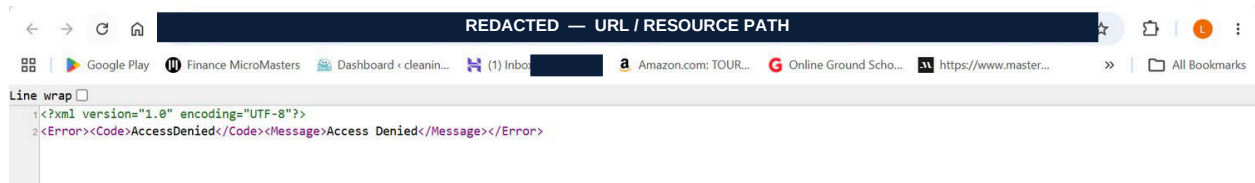


Figure 26. Layer one of the investigation — inspecting the rendered page source.

Reading exactly what the browser received in response. The first diagnostic principle is to confirm what actually arrived rather than what was expected. This rules out the front end and the redirect construction as the source of the fault.

Second layer — the DOM console

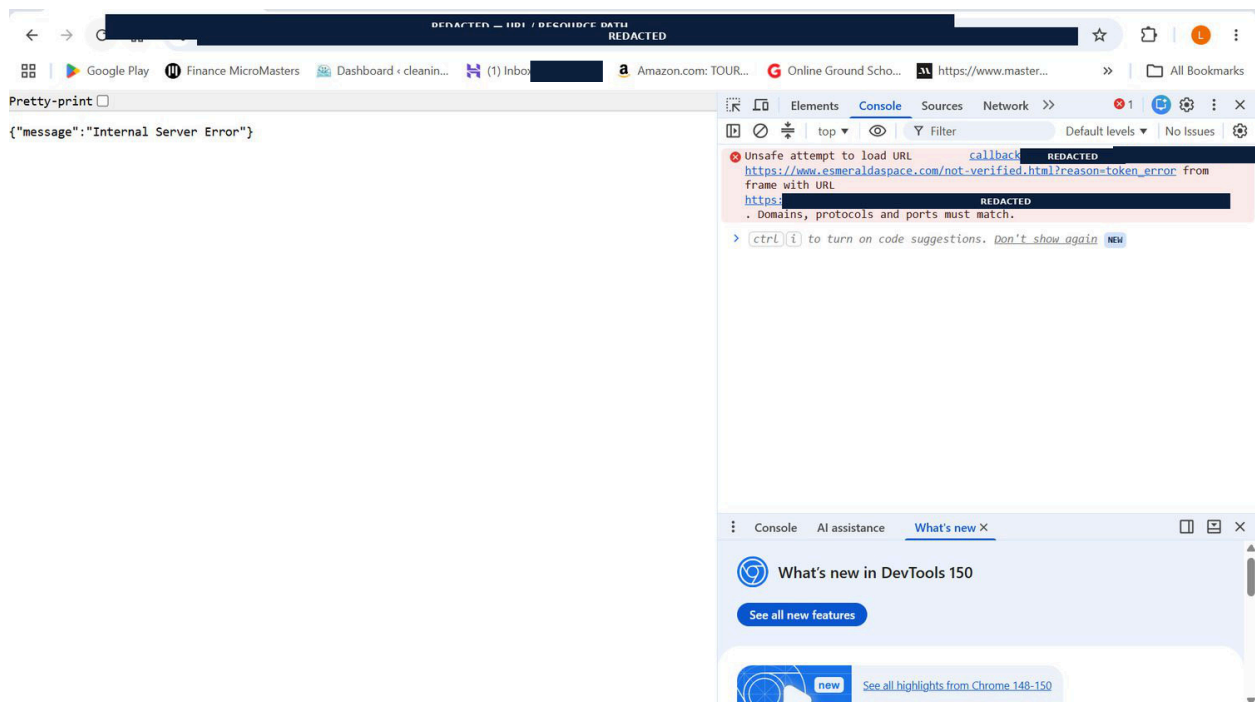


Figure 27. Layer two — interrogating the DOM console. (Endpoint and code redacted.)

Observing the full redirect chain and the precise request that failed. This confirms the request reached the gateway correctly formed. The fault is now isolated to the boundary between the gateway and the function — but the browser can see no further, which forces the investigation server-side.

Third layer — the CloudWatch logs

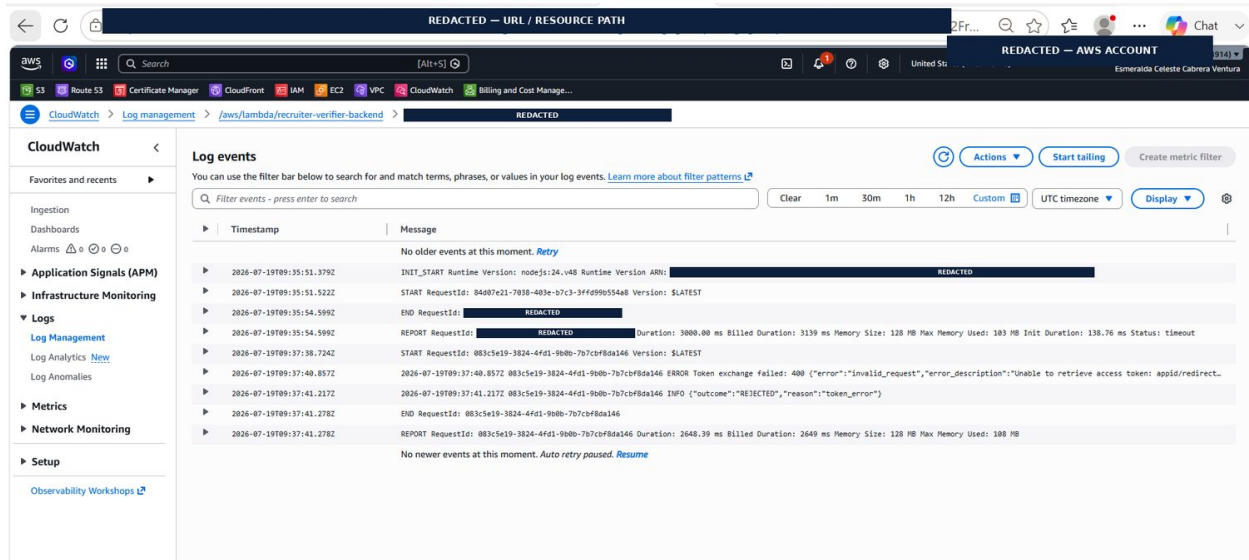


Figure 28. Layer three — CloudWatch logs, the authoritative record. (Identifiers redacted.)

Reading the function's own execution telemetry — the only place that records what the code actually did. This is where the investigation resolved. The execution report showed a duration pinned at exactly 3000.00 ms with a timeout status, and peak memory of 103 MB against a 128 MB ceiling. Neither fact is visible anywhere else in the stack.

Correcting the timeout and memory ceiling

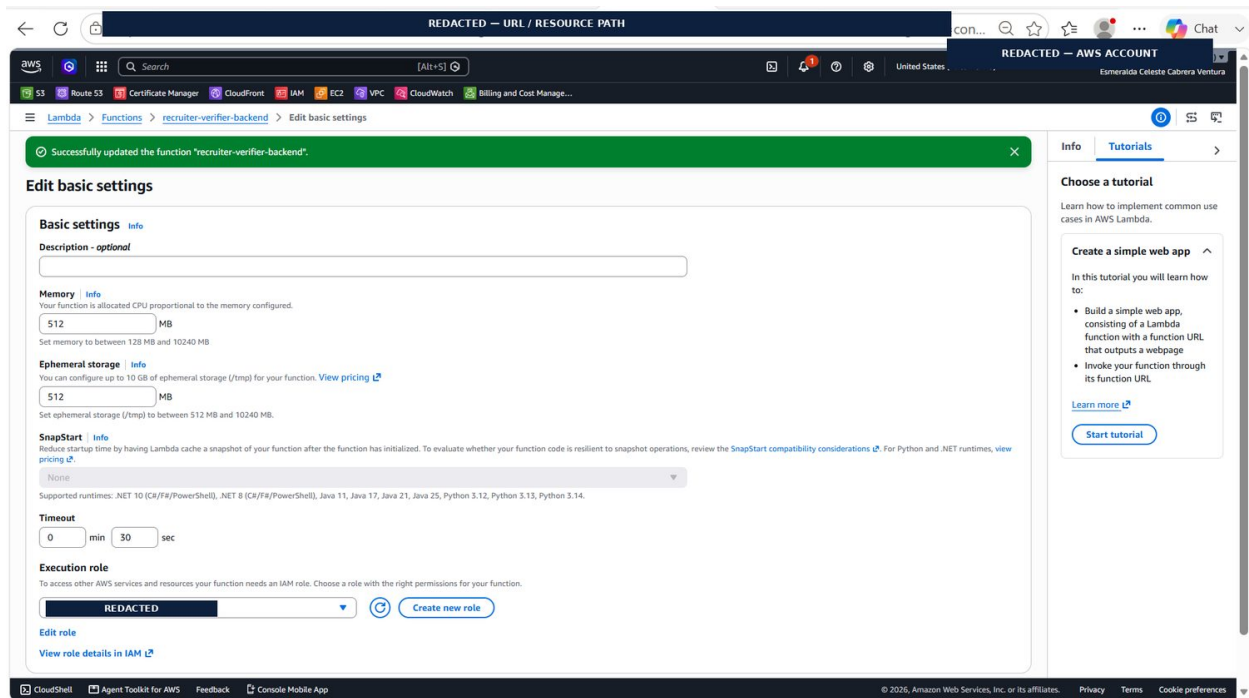


Figure 29. Correcting the root cause — timeout 3 s to 30 s, memory 128 MB to 512 MB.

Raising both limits based directly on the evidence in the logs. The two defaults compounded each other. Three seconds cannot cover a cold start plus a multi-hop conversation with an external identity provider. Worse, Lambda allocates CPU proportionally to configured memory — so the 128 MB setting was actively throttling the processor and lengthening the very cold start that was timing out. The function was starved of CPU and then killed for being slow.

Both corrections are recorded in the project's operations notes with the evidence that justified them — 'confirmed in CloudWatch', 'the function used 103 MB (80% of ceiling)' — so a future maintainer inherits the reasoning and not merely the values.

6.9 Static Hosting — Private Origin

With the backend correct, attention moved to delivery. The site is served from a bucket that is never directly addressable from the internet.

Creating the origin bucket

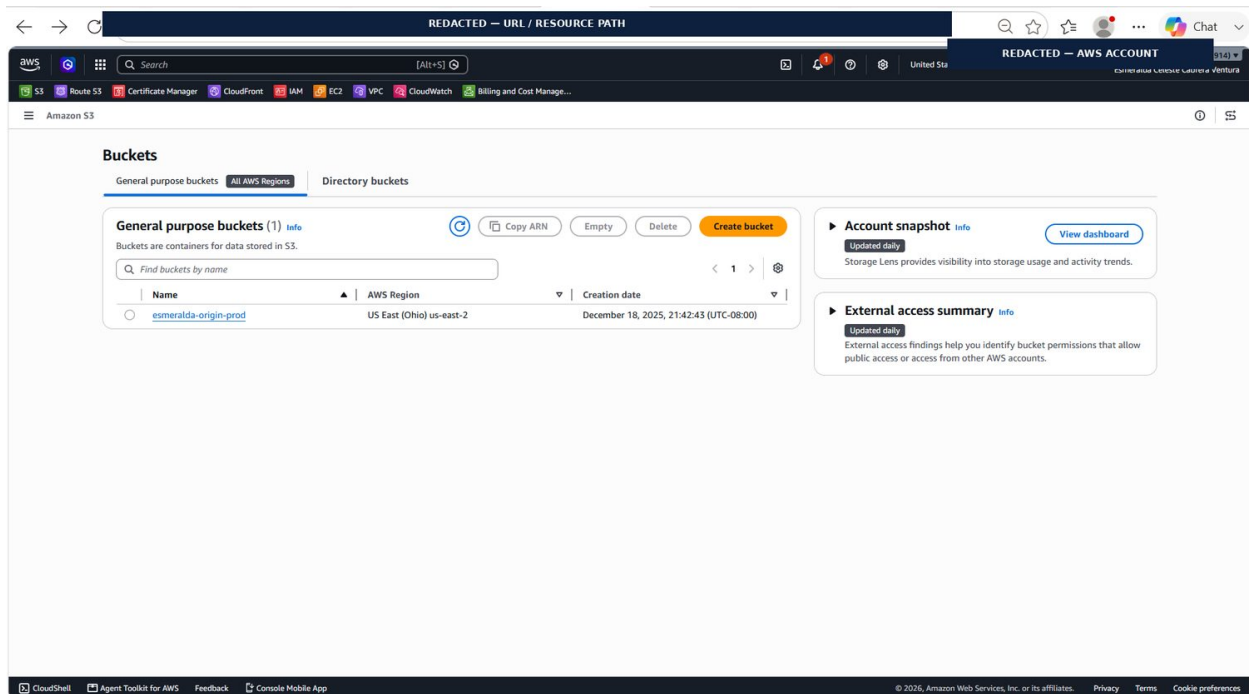


Figure 30. Creating the origin bucket.

Provisioning fresh object storage for the 2.0 build. A clean bucket avoids inheriting permissions or stale objects from the previous deployment. This bucket will hold the entire site but will never be publicly reachable.

Blocking public access

The screenshot shows the AWS Management Console 'Create bucket' page. The 'General configuration' section is active, showing the bucket is being created in the 'us-west-1' region. Under 'Bucket namespace', 'Global namespace' is selected. The 'Bucket name' field is redacted. In the 'Object Ownership' section, 'ACLs disabled (recommended)' is selected, which is the setting that blocks public access. The 'Copy settings from existing bucket' section is optional. The footer shows the AWS logo, navigation links, and copyright information.

Create bucket [info](#)

Buckets are containers for data stored in S3.

General configuration

AWS Region
US West (N. California) us-west-1

Bucket namespace
Choose the namespace where you want to create your bucket. [Learn more](#)

☒ **Global namespace**
By default, S3 creates general purpose buckets in the global namespace.

☐ **Account Regional namespace (recommended)**
General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name [info](#)
REDACTED

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#)

Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Format: s3://bucket/prefix

Object Ownership [info](#)
Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

Object Ownership

☒ **ACLs disabled (recommended)**
All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.

☐ **ACLs enabled**
Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Figure 31. Bucket configuration — public access controls.

Confirming that public access remains blocked. This is the control that makes the private-origin architecture real. The bucket is readable only by the CDN service principal; visitors never address it directly, so its contents cannot be enumerated or hot-linked.

Reviewing the storage settings

The screenshot shows the AWS S3 console 'Create bucket' page for the us-west-1 region. The page includes a warning about public access, a 'Bucket Versioning' section with 'Enable' selected, a 'Tags - optional' section with 'Add new tag' button, a 'Default encryption' section with 'Server-side encryption with Amazon S3 managed keys (SSE-S3)' selected, and an 'Advanced settings' section with a note about uploading files and folders. The 'Create bucket' button is at the bottom right.

Turning off block all public access might result in this bucket and the objects within becoming public. AWS recommends that you turn on block all public access, unless public access is required for specific and verified use cases such as static website hosting. [Learn more](#)

☒ I acknowledge that the current settings might result in this bucket and the objects within becoming public.

Bucket Versioning

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintentional user actions and application failures. [Learn more](#)

Bucket Versioning

☐ Disable

☒ Enable

Tags - optional

You can use bucket tags to analyze, manage and specify permissions for a bucket. [Learn more](#)

☒ You can use s3:ListTagForResource, s3:TagResource, and s3:UntagResource APIs to manage tags on S3 general purpose buckets for access control in addition to cost allocation and resource organization. To ensure a seamless transition, please provide permissions to s3:ListTagForResource, s3:TagResource, and s3:UntagResource actions. [Learn more](#)

No tags associated with this bucket.

[Add new tag](#)

You can add up to 50 tags.

Default encryption [info](#)

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type [info](#)

Secure your objects with two separate layers of encryption. For details on pricing, see [SSE-KMS pricing](#) on the [Amazon S3 pricing page](#).

☒ Server-side encryption with Amazon S3 managed keys (SSE-S3)

☐ Server-side encryption with AWS Key Management Service keys (SSE-KMS)

☐ Dual-layer server-side encryption with AWS Key Management Service keys (SSE-KMS)

Bucket Key

Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for SSE-KMS. [Learn more](#)

☐ Disable

☒ Enable

Advanced settings

☒ After creating the bucket, you can upload files and folders to the bucket, and configure additional bucket settings.

[Cancel](#) [Create bucket](#)

Figure 32. Bucket configuration continued.

Reviewing the remaining storage settings before any object is uploaded. Storage-level properties are far easier to establish correctly at creation than to retrofit once a live site depends on the bucket.

Uploading the site build

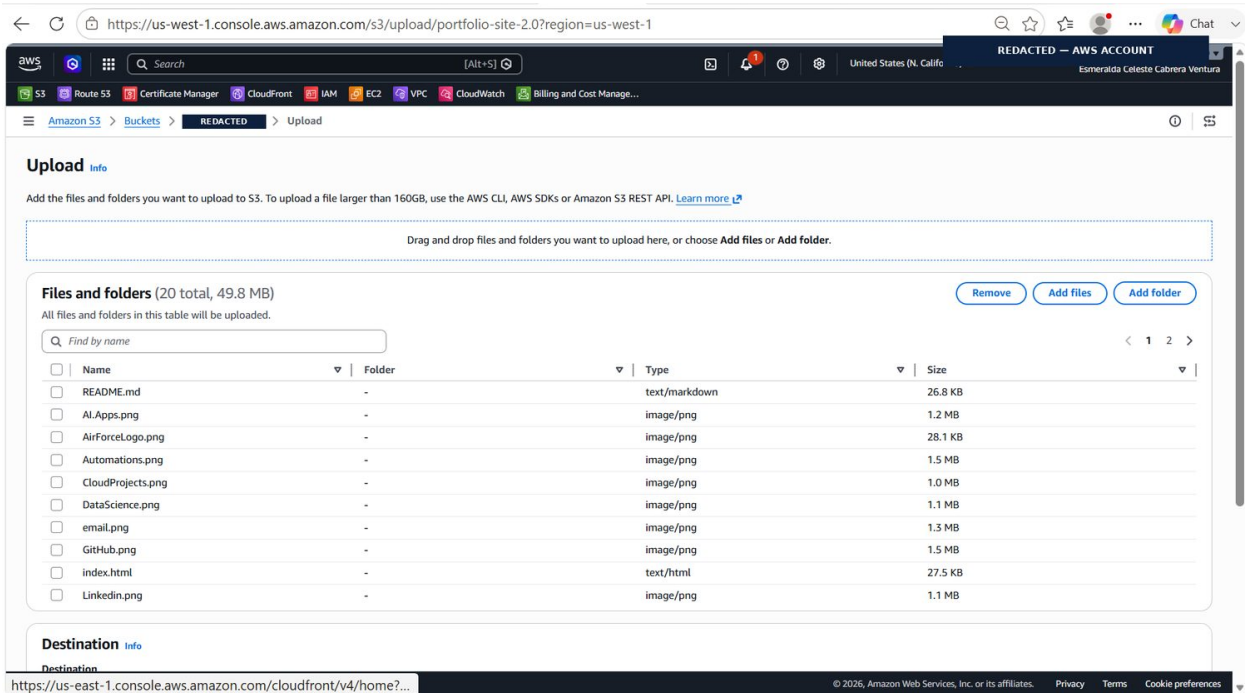


Figure 33. Uploading the site build.

Transferring the front-end assets, including the two new outcome pages. The outcome pages are not optional decoration — they are the destinations the Lambda redirects to. If they are absent, every verification attempt terminates in a storage error regardless of whether the visitor passed.

Uploading the media assets

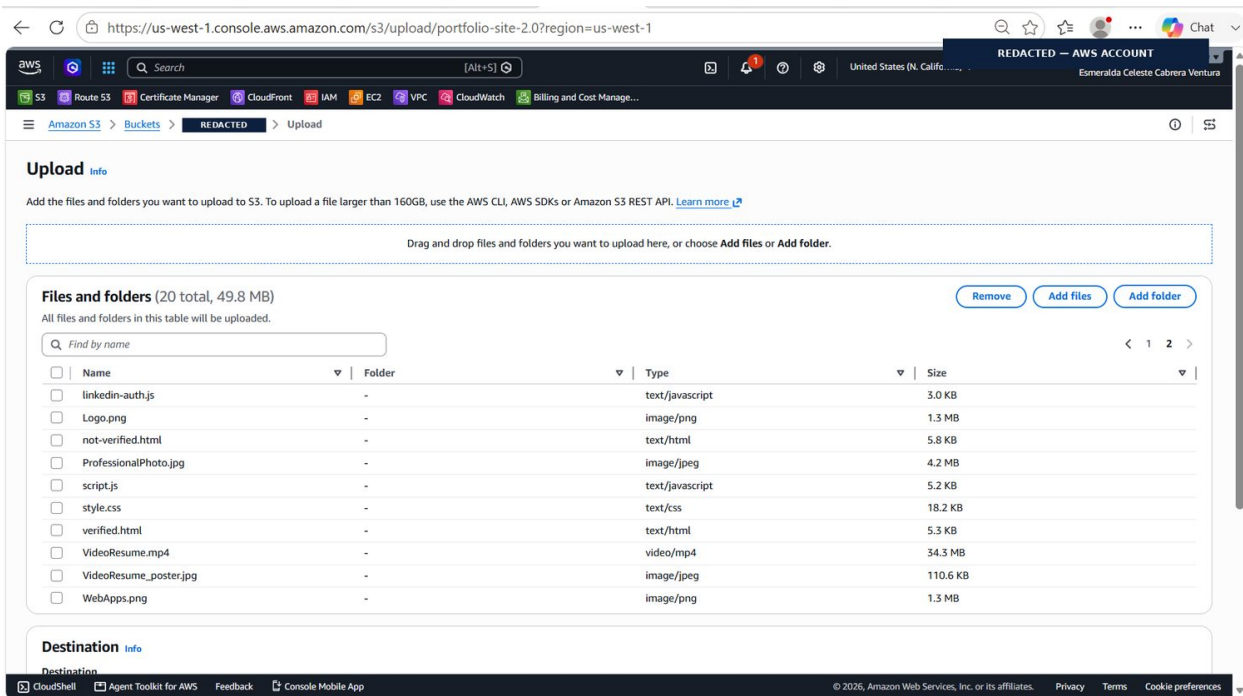


Figure 34. Upload continued — media assets including the AI avatar video.

Uploading the larger binary assets, including the generated video resume. Serving heavy media from the CDN rather than the origin keeps latency low globally and prevents large transfers from repeatedly hitting the storage bucket.

The complete object set

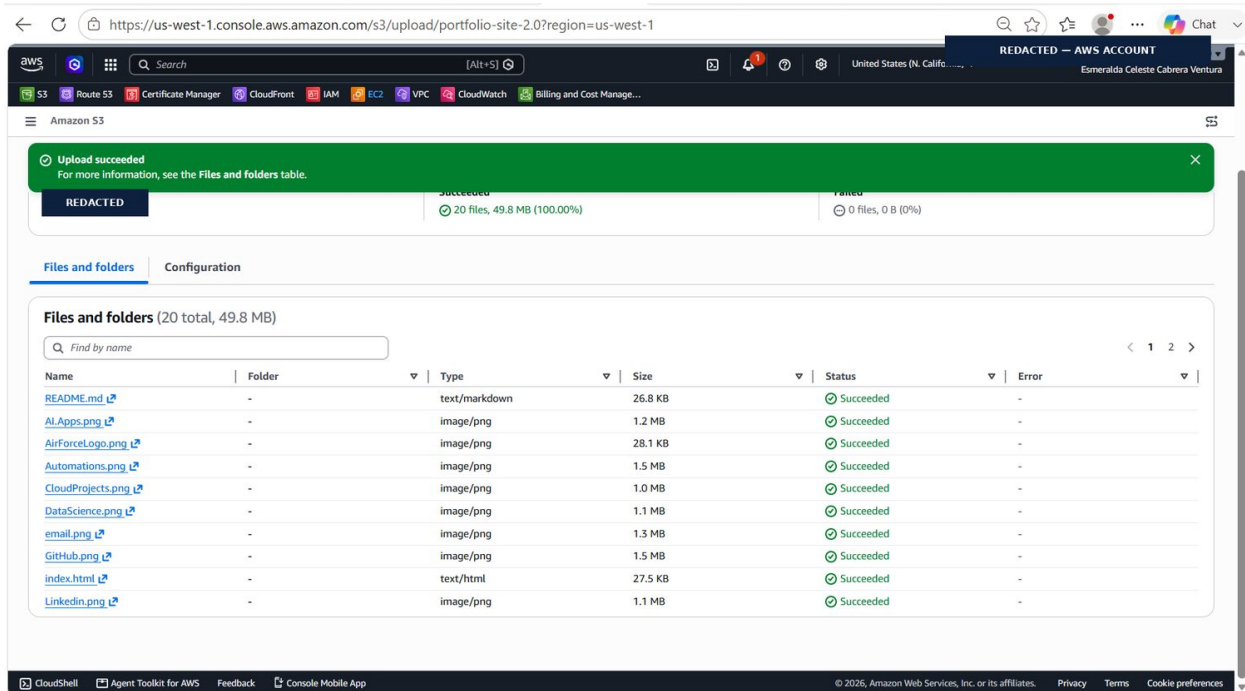


Figure 35. Upload confirmed — the complete site resident in the private origin.

Verifying the full object set landed successfully. A missing object here surfaces later as a confusing access-denied error rather than a clean not-found, because a bucket policy without list permission cannot distinguish the two. Confirming the manifest now prevents a misleading failure later.

6.10 Global Delivery — CloudFront

The distribution was repointed at the new origin. Two details matter more than they look: origin access keeps the bucket private, and the cache behaviour must forward query strings — because the entire approval hand-off travels as one.

Locating the CDN distribution

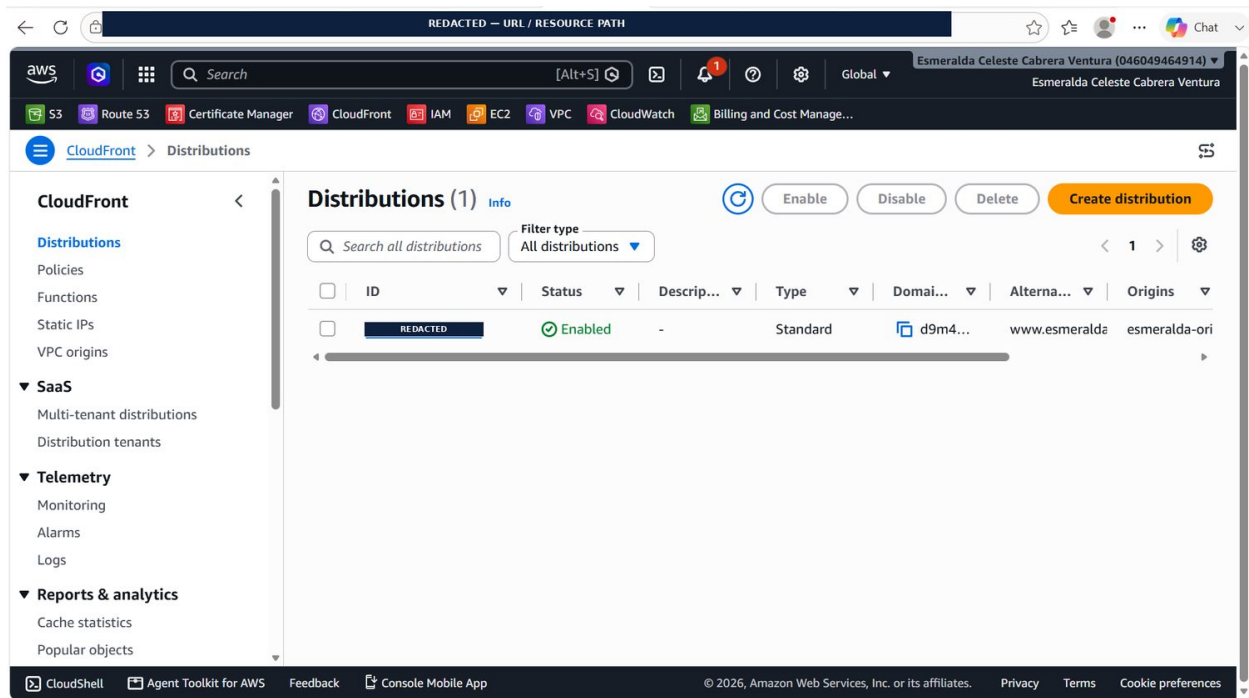


Figure 36. Locating the existing distribution serving the custom domain. (Identifier redacted.)

Identifying the CDN distribution already bound to the site's domain and TLS certificate. Reusing the existing distribution preserves DNS and certificate configuration. Creating a new one would mean re-validating the certificate and re-pointing DNS — unnecessary risk for what is only an origin change.

Repointing the origin

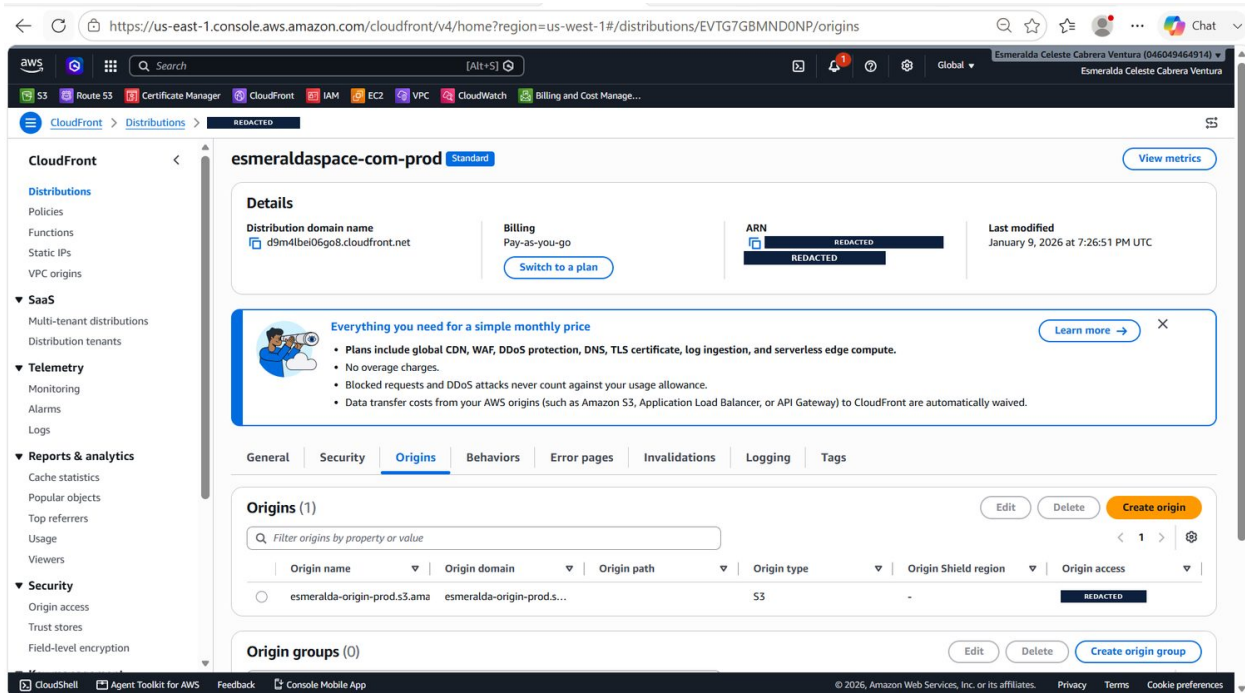


Figure 37. Repointing the distribution origin at the new bucket. (Identifiers and ARNs redacted.)

Updating the origin so the CDN fetches from the new storage bucket. This single change swaps the entire site over atomically at the edge, with no DNS propagation delay and no window where visitors see a half-deployed site.

Configuring origin access control

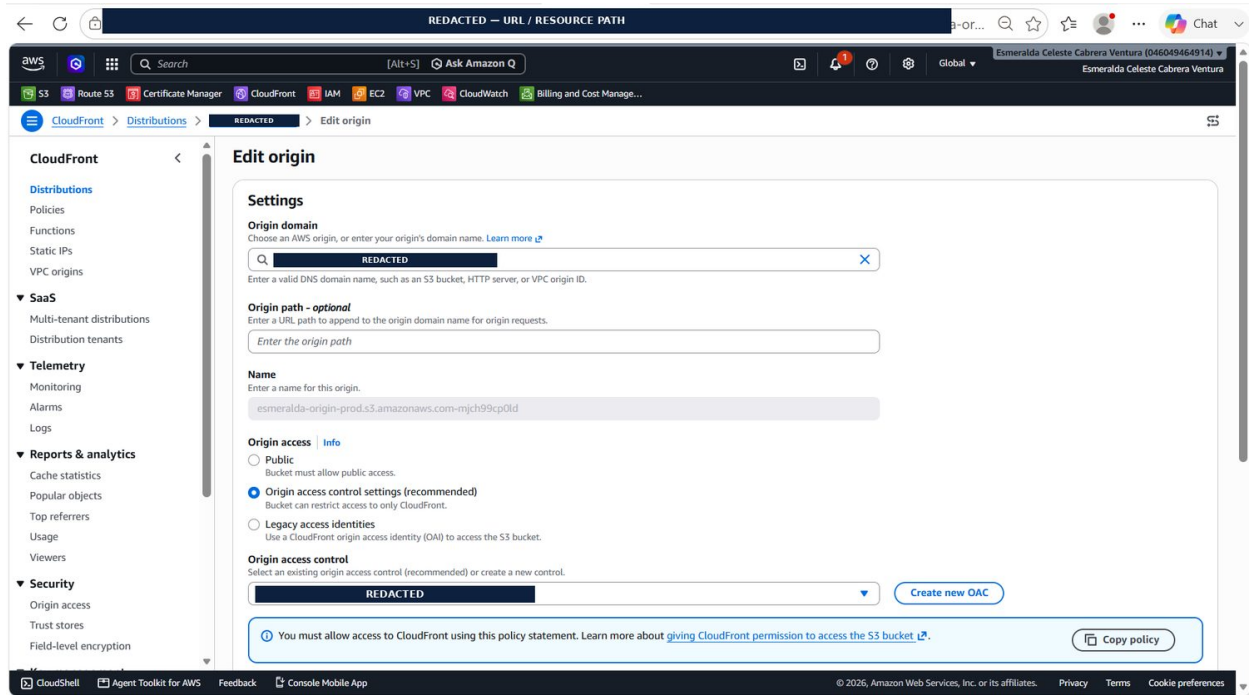


Figure 38. Origin settings continued — access configuration keeping the bucket private. (Identifiers redacted.)

Configuring Origin Access Control so the CDN authenticates to the bucket as a service principal. This is what allows the bucket to stay completely private while still being globally served. The CDN holds the only read relationship with the origin.

The origin change propagates

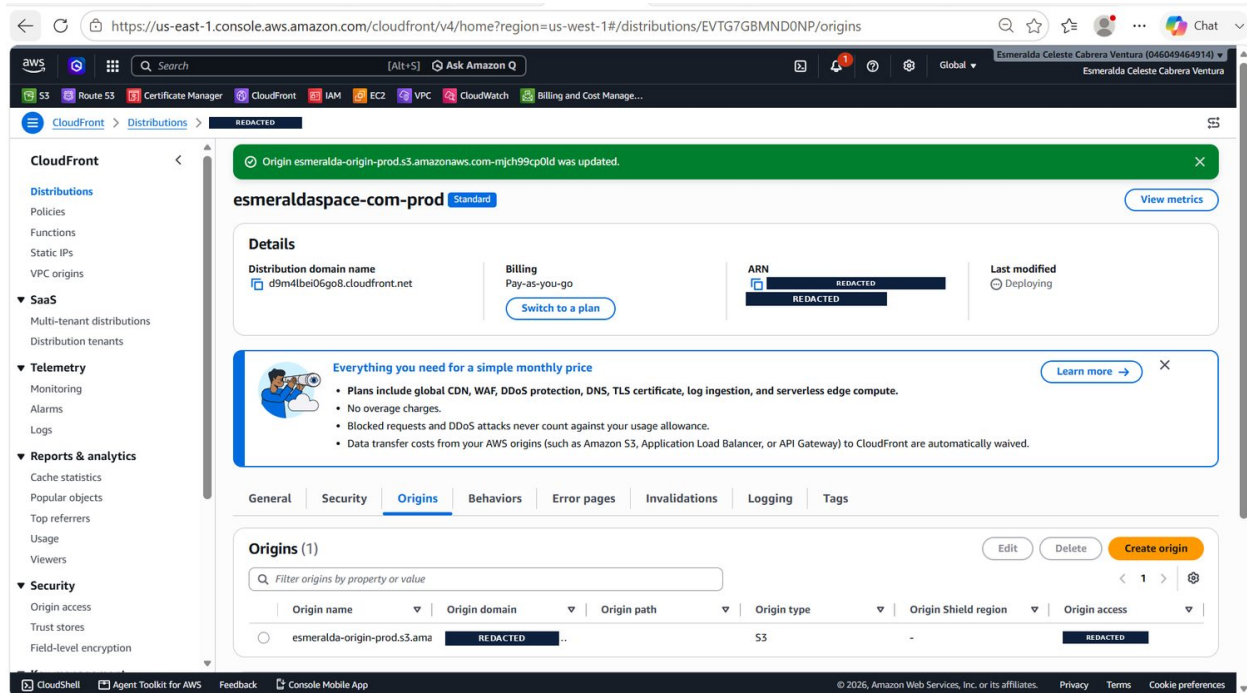


Figure 39. Origin update completed. (Identifiers and ARNs redacted.)

The origin change is saved and propagating to edge locations. Confirms the distribution and the new bucket are correctly bound before cache behaviour is addressed.

Invalidating the edge caches

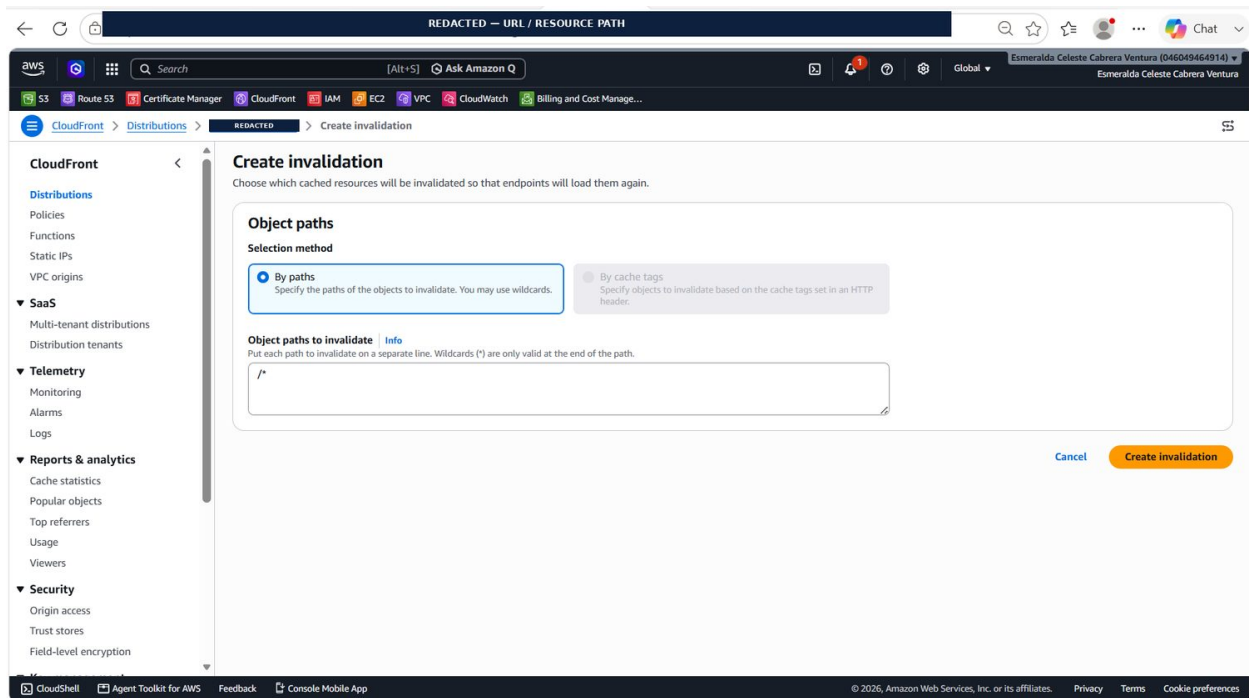


Figure 40. Creating a full cache invalidation.

Instructing every edge location to discard its cached copies of the site. Without this the CDN keeps serving the previous build from cache — visitors would see the old page and receive 404s for the newly added outcome pages, because the edge has no knowledge that new objects exist at the origin.

Invalidation confirmed

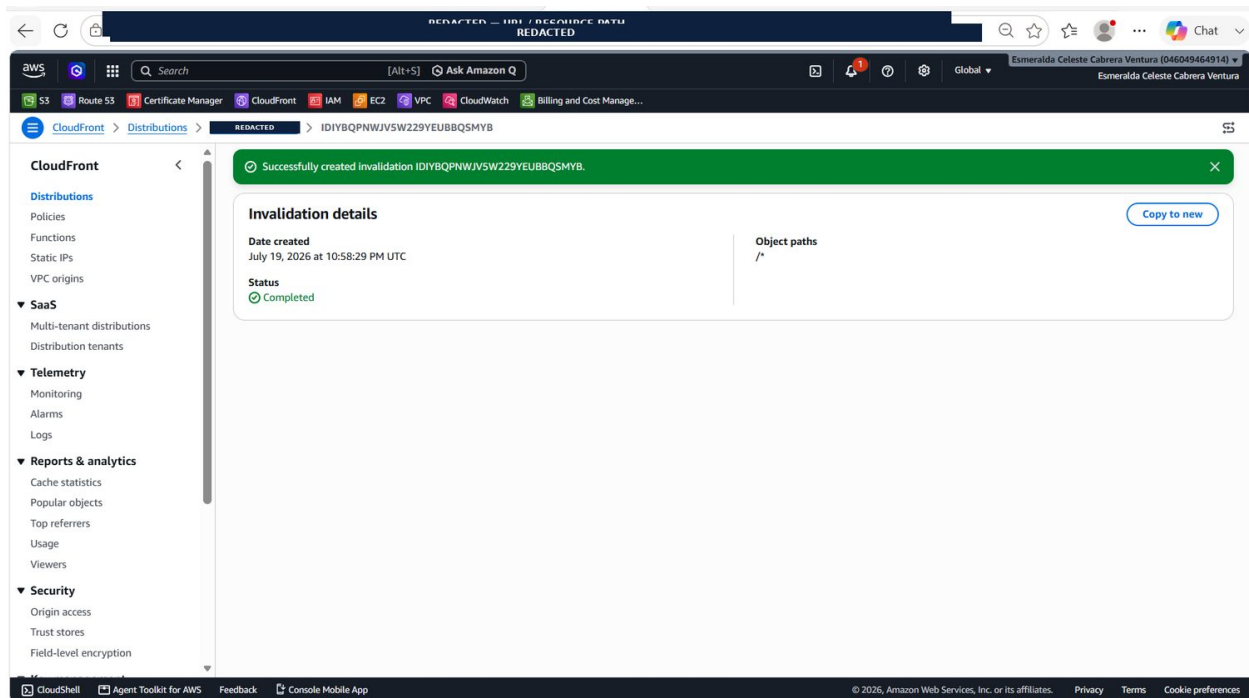


Figure 41. Invalidation confirmed and propagating. (Identifier redacted.)

The invalidation request is accepted and executing across the edge network. This is the step that makes a deployment actually visible to the public. Skipping it is one of the most common causes of a 'deployed but unchanged' site.

6.11 Origin Access Policy

The bucket policy is the control that makes the private-origin design real rather than merely intended.

Authoring the origin access policy

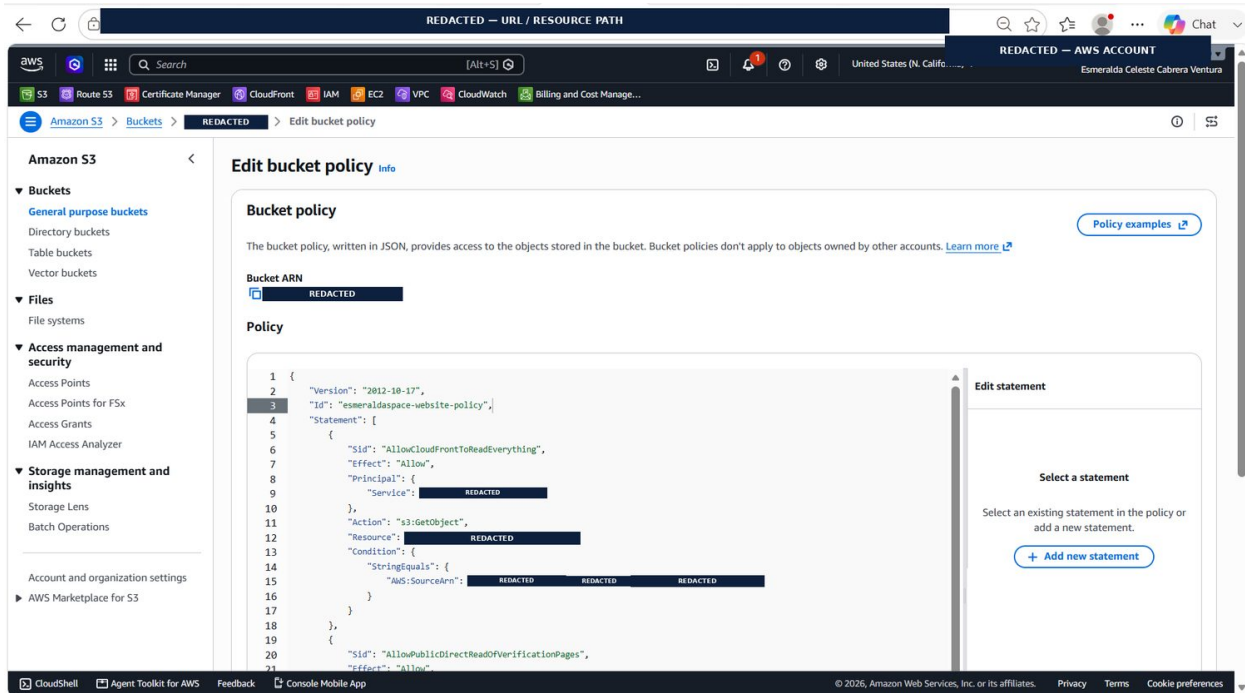


Figure 42. Authoring the bucket policy. (Account number, bucket name and ARNs redacted.)

Granting object-read permission to the CDN service principal, conditioned on the specific source distribution ARN. The condition is the security control. Without it, the policy would permit the CDN service to read the bucket on behalf of *any* distribution, including one created by an attacker in a different account. Scoping to a single distribution ARN closes that hole.

The policy takes effect

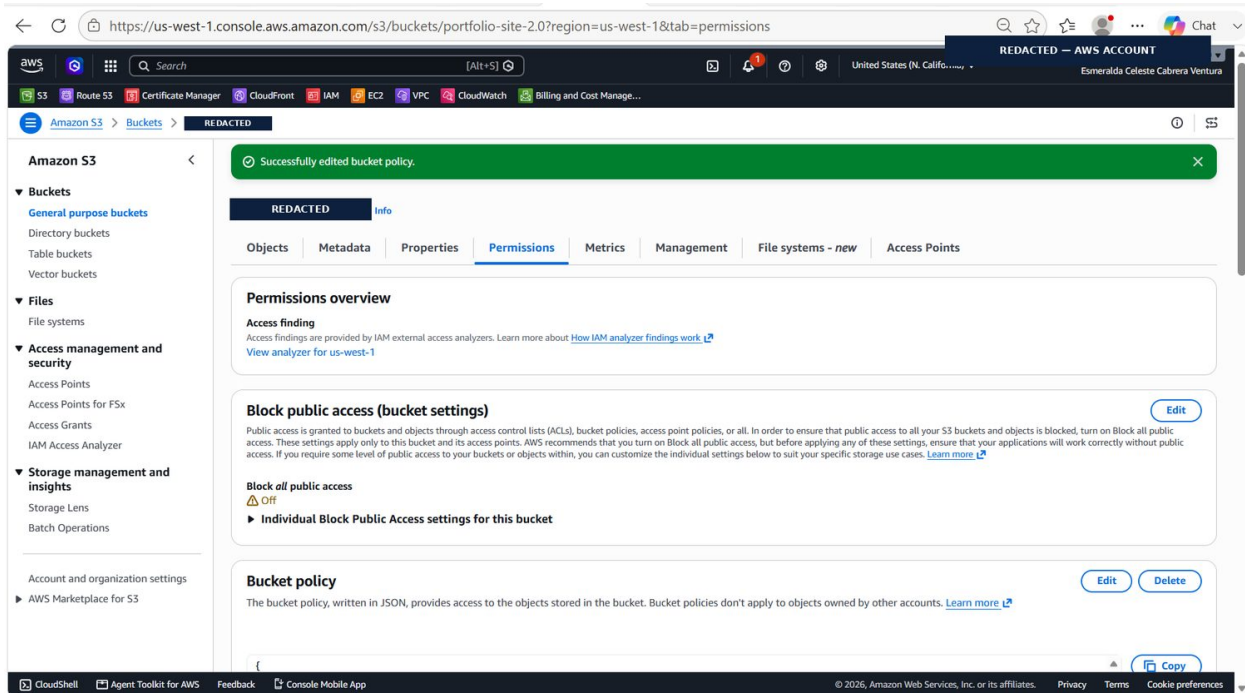


Figure 43. Policy applied successfully.

The bucket policy is saved and in force. The private-origin architecture is now genuinely live rather than merely intended — the bucket is unreachable except through the one distribution that is allowed to read it.

6.12 Wiring the Outcome Routing

The function must know exactly where to send an approved visitor and a rejected one.

Capturing the success destination

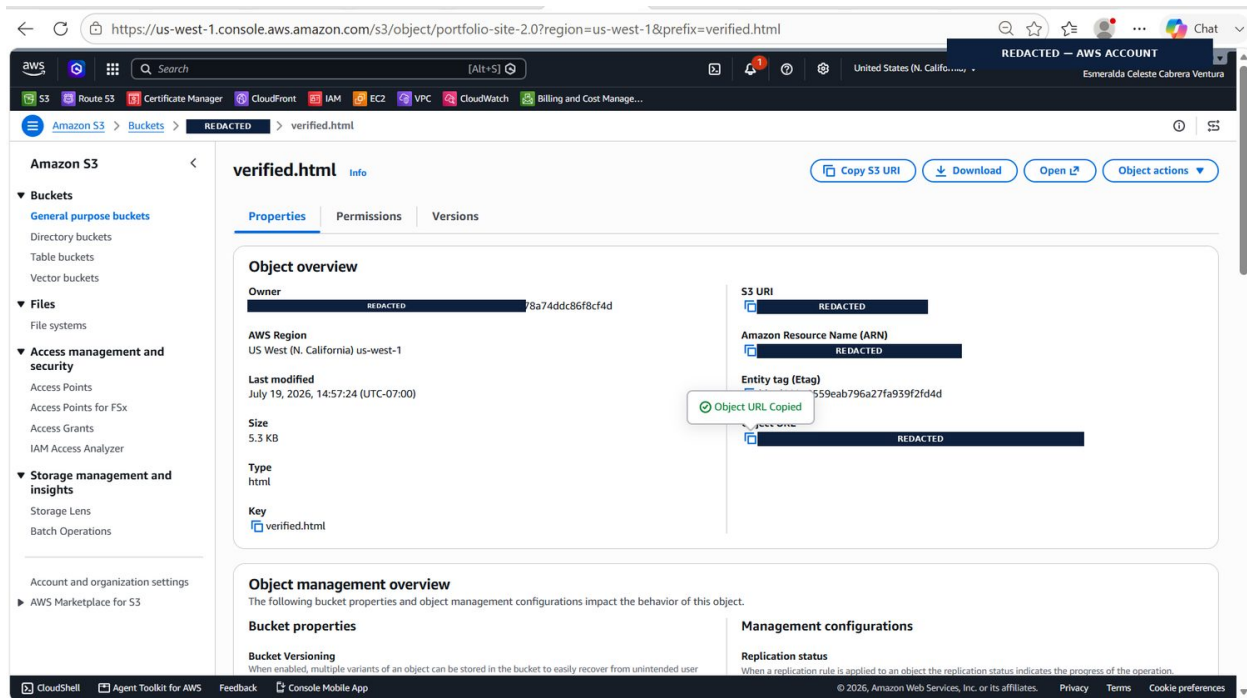


Figure 44. Capturing the success page URL. (Bucket name and object URL redacted.)

Retrieving the canonical address of the page an approved visitor is sent to. The function has to know its destinations exactly. This value is about to be written into the function's configuration as the success redirect target.

Capturing the failure destination

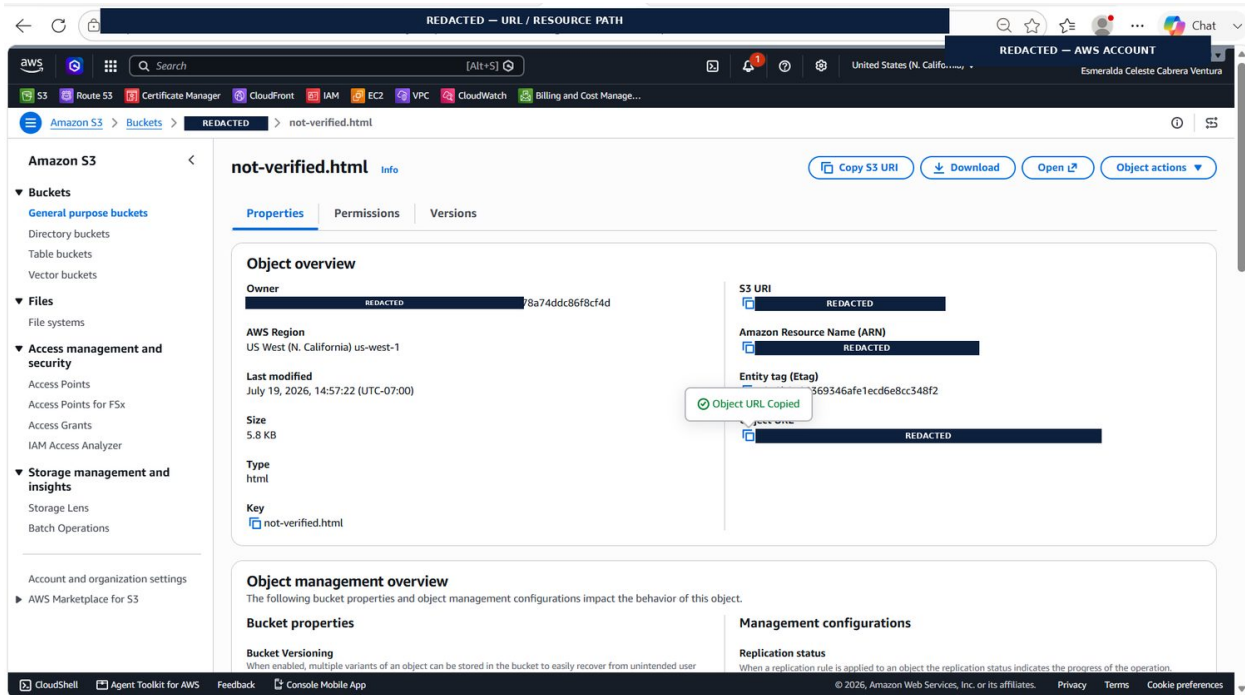


Figure 45. Capturing the failure page URL. (Bucket name and object URL redacted.)

Retrieving the address of the rejection page. Rejection is a first-class outcome in this design, not an error state. It gets its own dedicated destination, which is what makes it possible to deliver a specific, useful explanation to each rejected visitor.

Wiring the outcome destinations

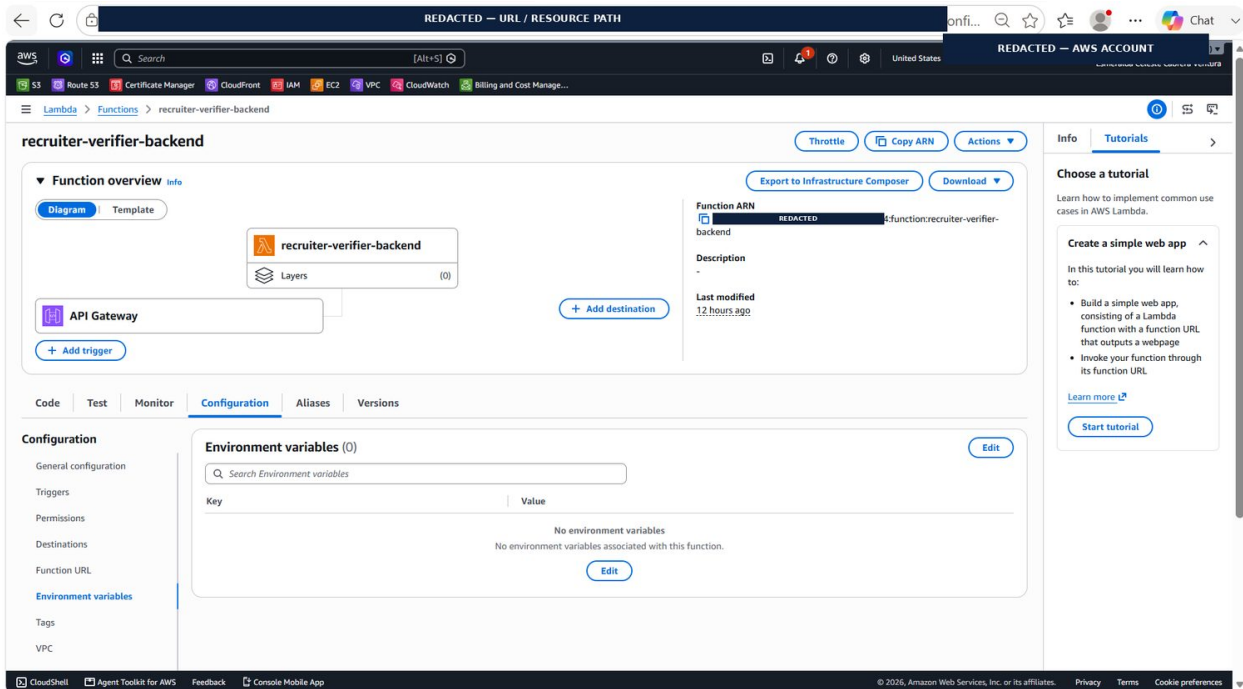


Figure 46. Updating the function configuration with the outcome destinations. (Identifiers redacted.)

Wiring the success and failure URLs into the function so it knows where to route each verdict. This is the step that closes the loop between the decision engine and the pages a visitor actually sees. Until it is done, the function can decide correctly but has nowhere to send anyone.

Completing the configuration

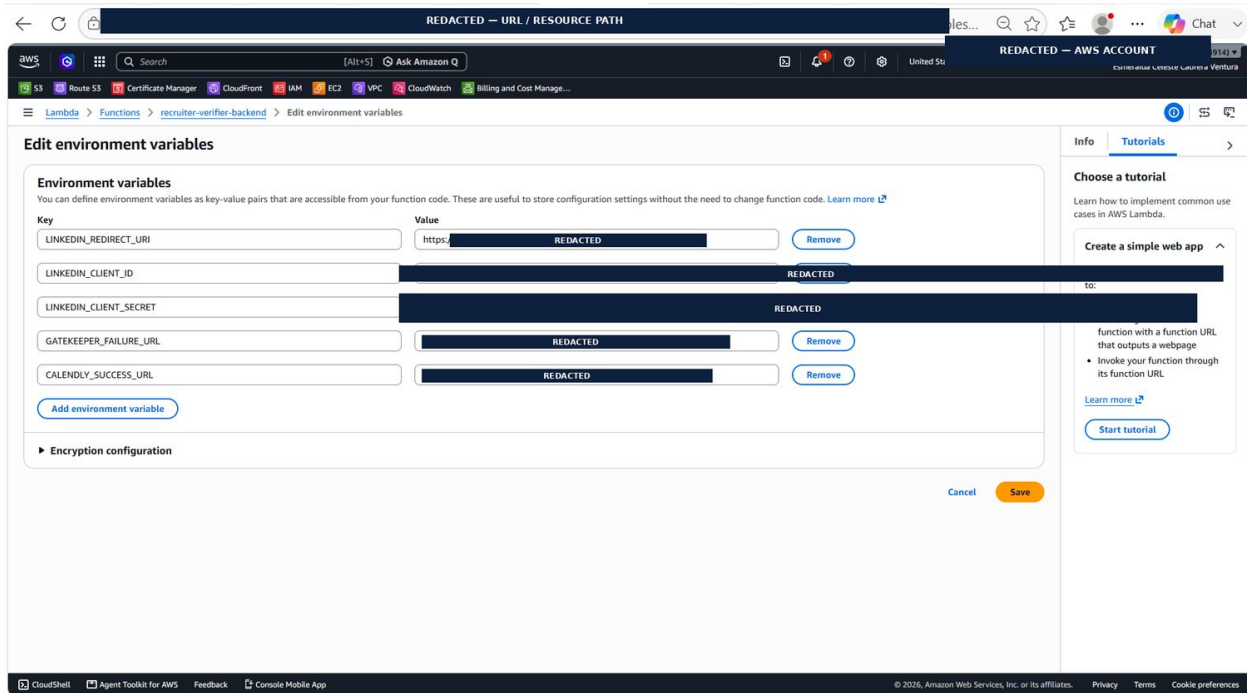


Figure 47. Configuration continued. (Client secret, endpoints and object URLs redacted.)

The remaining configuration values, including the identity credentials. Seeing the client secret sitting alongside ordinary configuration is exactly why this report recommends migrating these values to a managed secret store with automatic rotation. Configuration and credentials deserve different handling.

The configuration saved

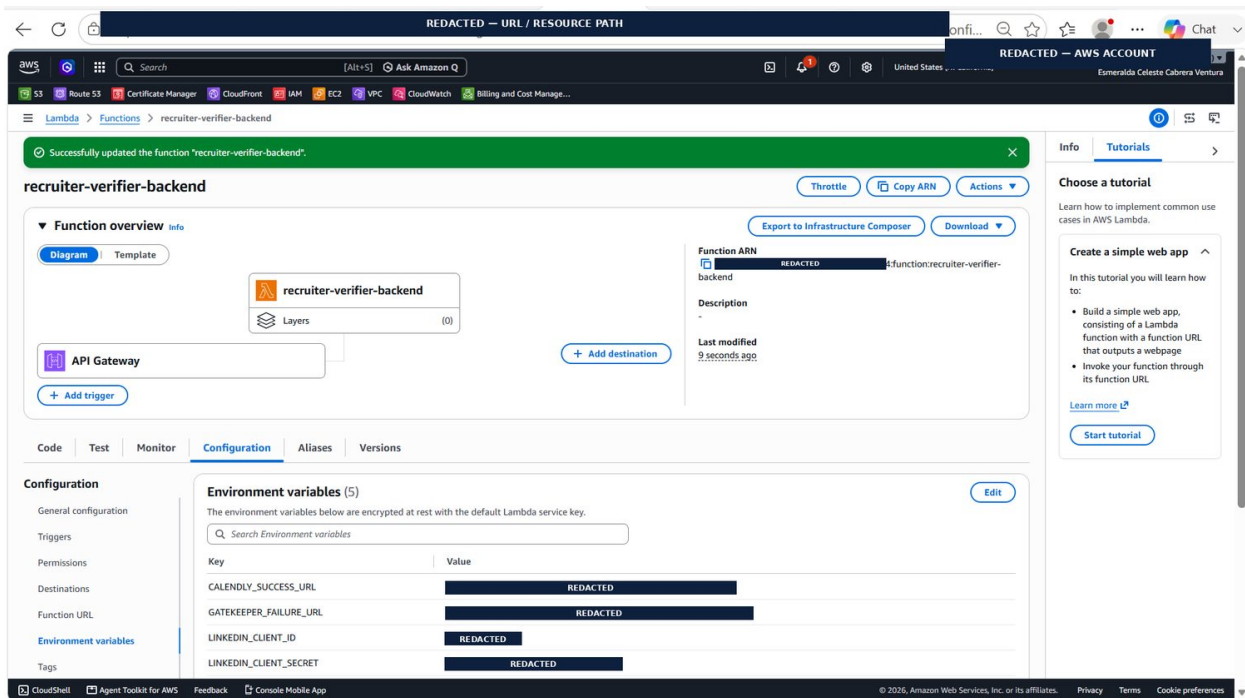


Figure 48. Configuration completed and saved. (Credentials and object URLs redacted.)

The function is fully configured and ready for a live end-to-end test. Every component is now connected — front end, identity provider, gateway, function, outcome pages and CDN. The next screenshots test whether the chain holds under a real visitor.

6.13 End-to-End Validation on the Live System

The final sequence exercises the entire pipeline exactly as a real recruiter would experience it — from public site to a confirmed booking in an inbox.

The upgraded site, live

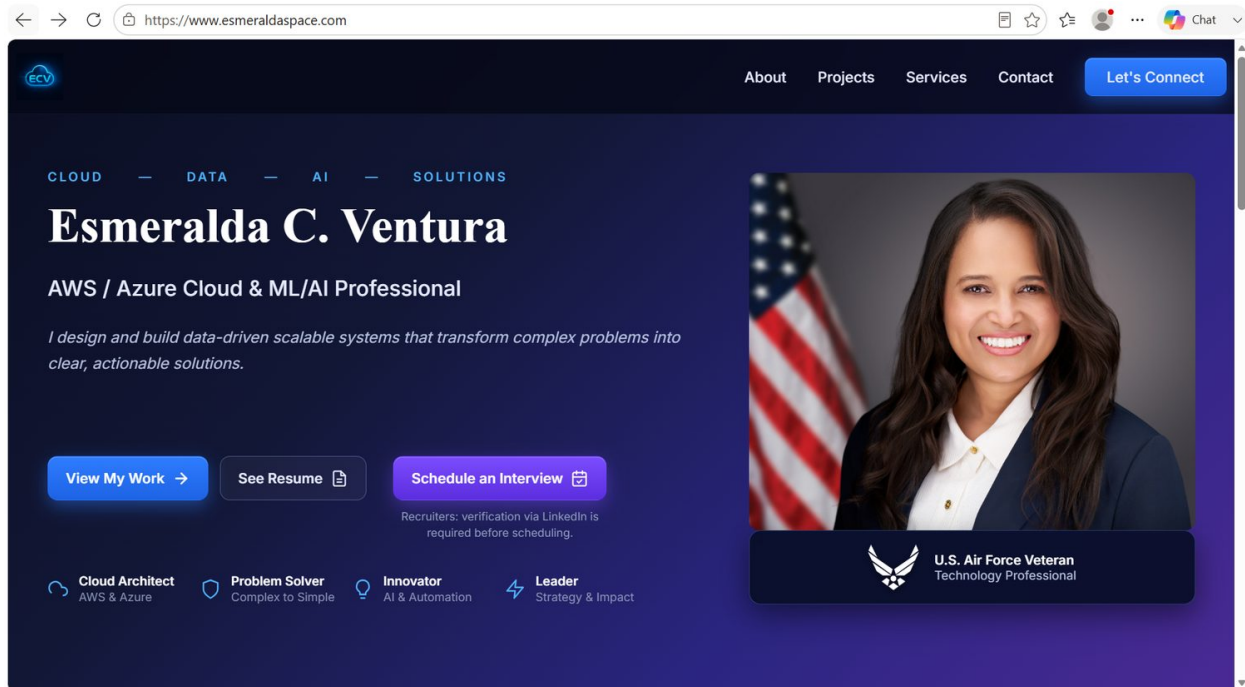
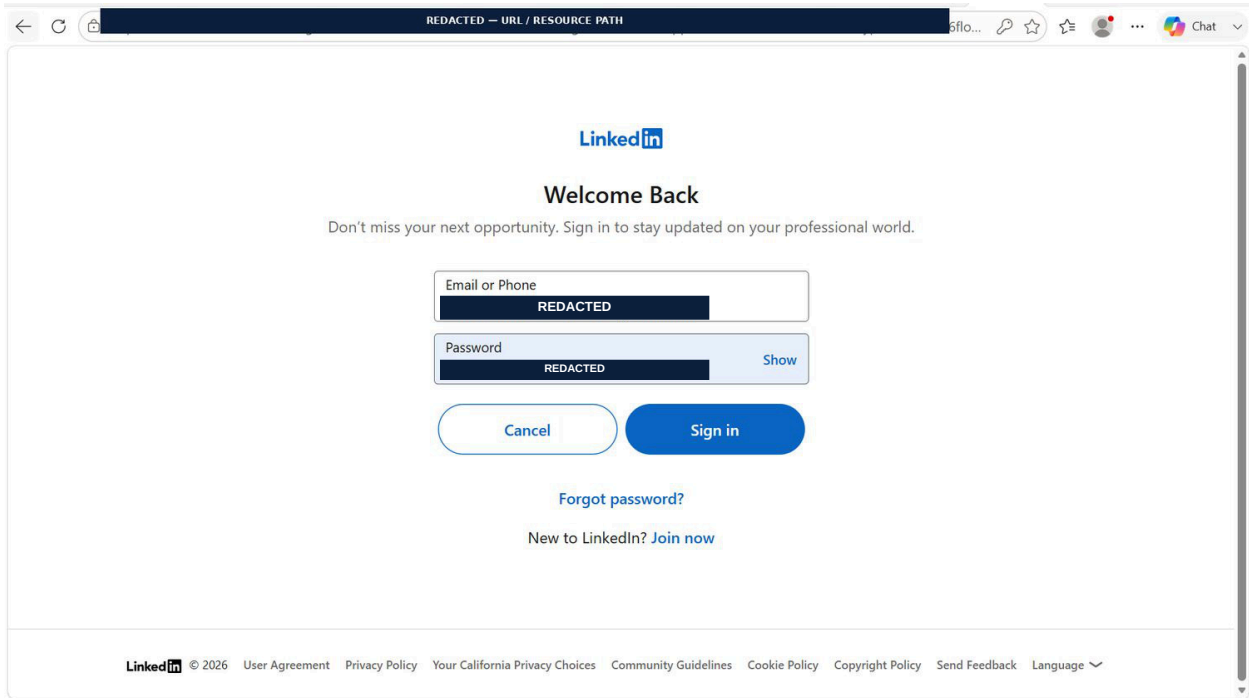


Figure 49. The upgraded site live at the custom domain over the CDN.

The new build is serving publicly through the distribution. Confirms the origin repo, the bucket policy and the invalidation all worked together correctly. A failure in any one of the three would surface here as a stale page or a broken asset.

The identity flow begins



The screenshot shows a web browser window with the LinkedIn login page. The browser's address bar displays "REDACTED — URL / RESOURCE PATH". The page content includes the LinkedIn logo, a "Welcome Back" heading, and a message: "Don't miss your next opportunity. Sign in to stay updated on your professional world." Below this is a sign-in form with two input fields: "Email or Phone" and "Password", both containing the text "REDACTED". To the right of the password field is a "Show" link. Below the input fields are two buttons: "Cancel" and "Sign in". Under the "Sign in" button are two links: "Forgot password?" and "New to LinkedIn? Join now". The footer of the page contains the LinkedIn logo, copyright information "© 2026", and a series of links: "User Agreement", "Privacy Policy", "Your California Privacy Choices", "Community Guidelines", "Cookie Policy", "Copyright Policy", "Send Feedback", and "Language".

Figure 50. The Schedule an Interview control initiating the identity flow.

The button generates a cryptographic CSRF nonce, stores it in session storage, and hands the browser to the identity provider. The nonce is what allows the round trip to be tied back to the browser tab that started it, defending the flow against cross-site request forgery.

Verification passes

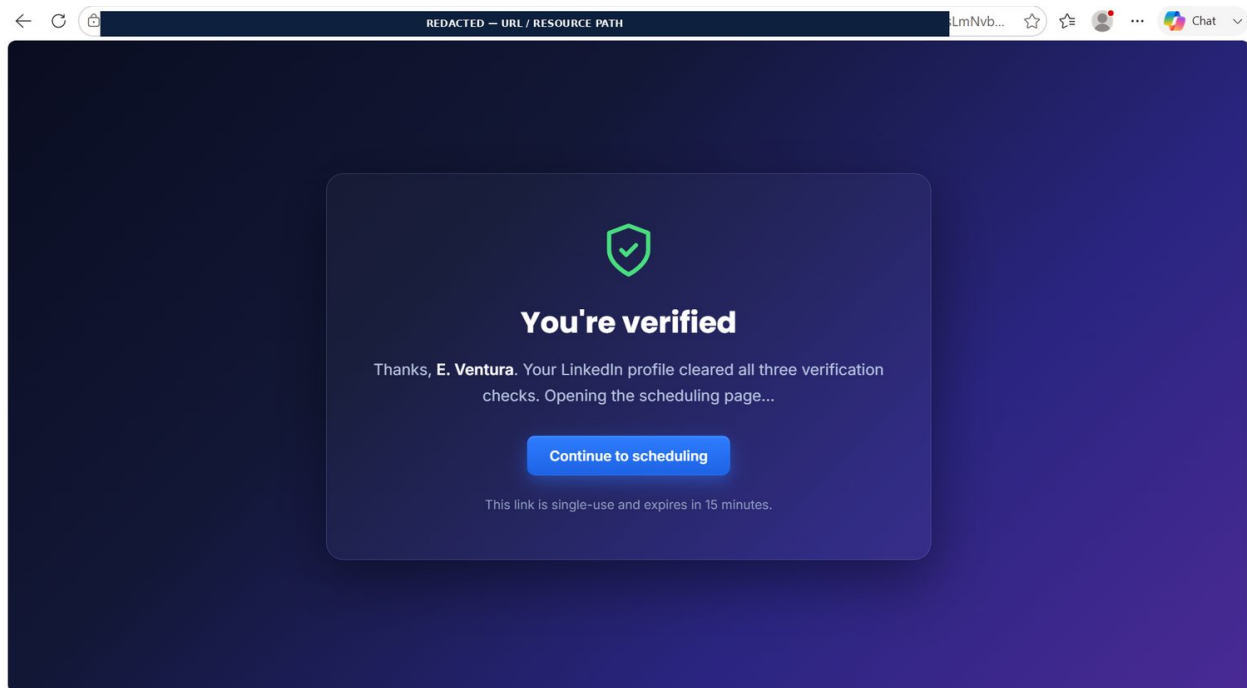


Figure 51. Verification passed — the policy engine approves the visitor. (Token and identifiers redacted.)

The rebuilt engine evaluates the five tests, reaches an approval verdict, and issues a signed, expiring token. This is the exact scenario that returned a rejection for one hundred percent of visitors before the three-state redesign. It is the single most important screenshot in the report.

Hand-off to the calendar

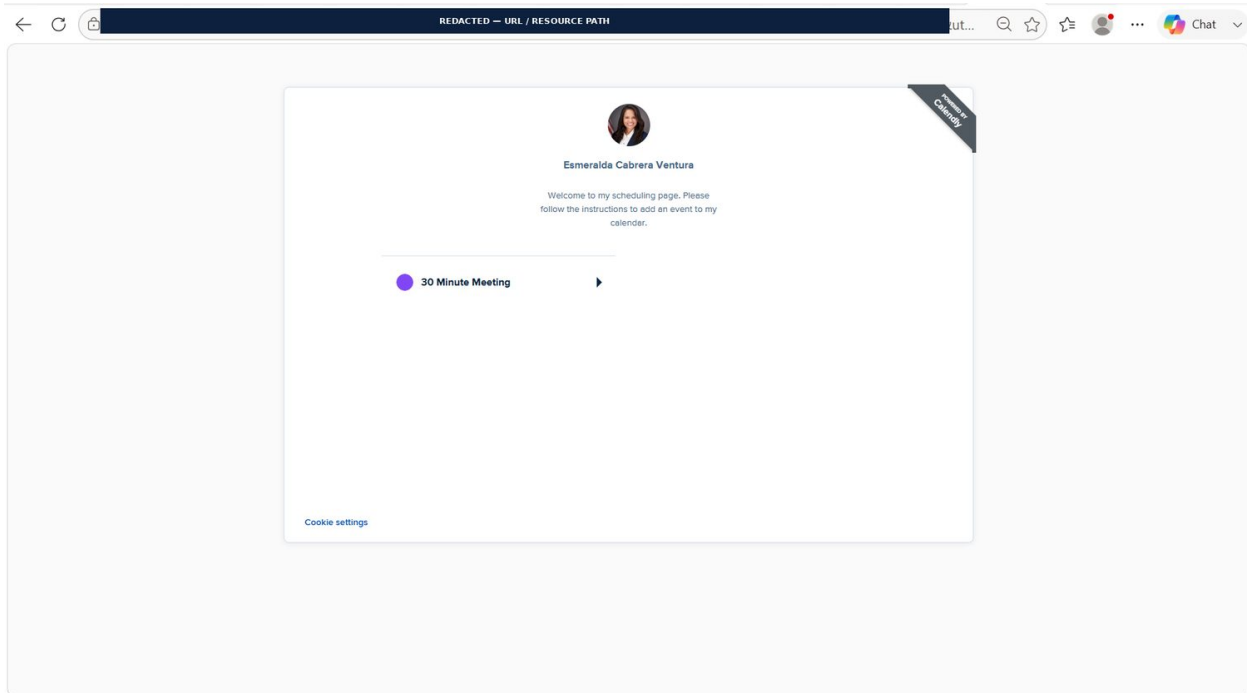


Figure 52. The approved visitor is forwarded to the protected booking calendar.

The success redirect delivers the verified visitor onward to the scheduling resource. Demonstrates the complete hand-off working — identity established, policy satisfied, token minted, access granted.

Live availability, visible

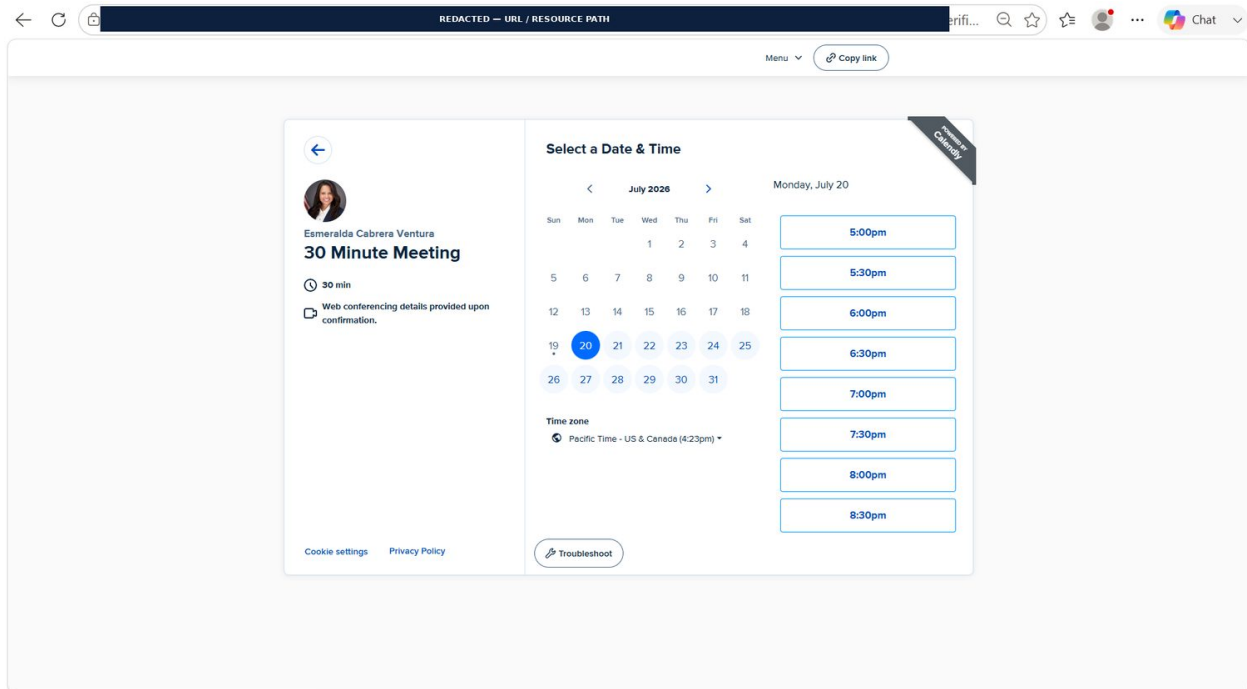


Figure 53. The verified visitor can see live availability.

The protected calendar renders real, synchronised availability. Confirms the resource behind the gate is fully functional for an approved visitor, and that the calendar integration configured at the start is working as intended.

Browsing the available slots

The screenshot shows a web browser window with a dark header bar containing navigation icons and a 'Chat' button. The main content area displays a '30 Minute Meeting' form for Esmeralda Cabrera Ventura. The form is divided into two columns. The left column contains a back arrow, a profile picture, the name 'Esmeralda Cabrera Ventura', the title '30 Minute Meeting', a '30 min' duration, a note about web conferencing details, the date and time '8:00pm - 8:30pm, Monday, July 20, 2026', and the time zone 'Pacific Time - US & Canada'. At the bottom of this column are links for 'Cookie settings' and 'Privacy Policy'. The right column is titled 'Enter Details' and includes a 'Name' field with the value 'E. Ventura', an 'Email' field with a masked address, an 'Add guests' button, a text area for additional details, a disclaimer about terms and privacy, and a blue 'Schedule Event' button. A 'Copy link' button is located at the top right of the form area.

Figure 54. Availability continued.

Browsing further into the available slots. Verifies that the entire booking interface, not merely its landing view, is reachable and operational after verification.

Selecting a meeting slot

REDACTED — URL / RESOURCE PATH

Menu Copy link

[Cookie settings](#) [Privacy Policy](#)

Enter Details

Name *

E. Ventura

Email *

Add guests

Please share anything that will help prepare for our meeting.

Scheduling Meeting Test

By proceeding, you confirm that you have read and agree to [Calendly's Participant Terms](#) and [Privacy Notice](#).

Schedule Event

Figure 55. Selecting and confirming a meeting slot.

Completing the booking form as a verified visitor would. Tests the actual objective of the system rather than just the security boundary — a gate that blocks abuse but also obstructs legitimate booking would be a failure.

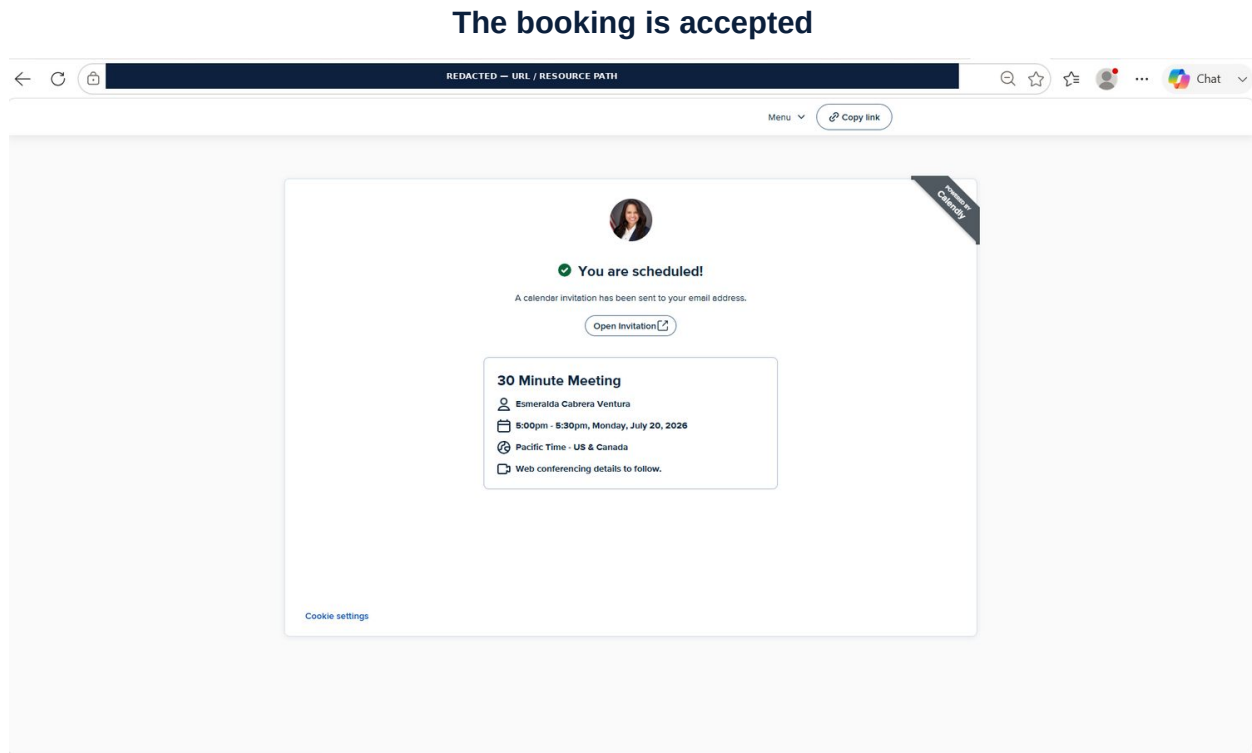


Figure 56. *The booking is accepted — the full chain has held end to end.*

The scheduling platform confirms the meeting. Every component in the architecture has now been exercised in sequence by a single real request, from anonymous click to confirmed appointment.

The invitation arrives

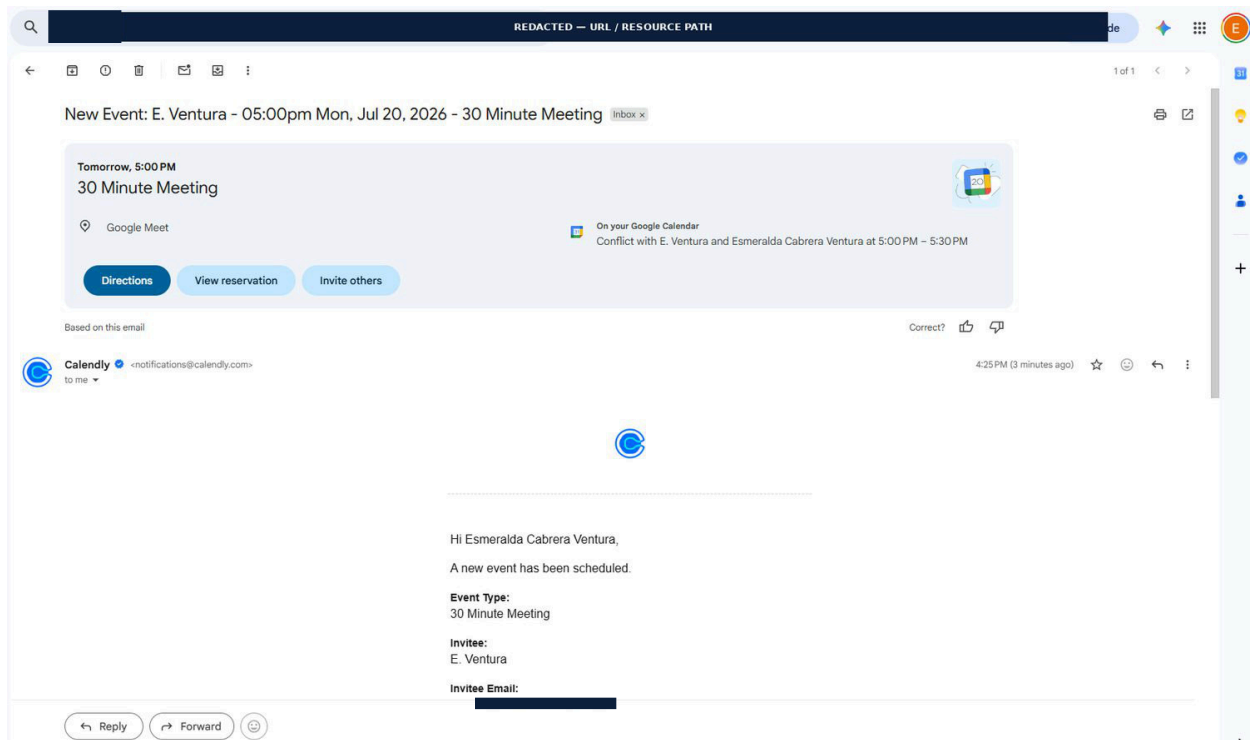


Figure 57. The calendar invitation arrives in the visitor's inbox.

The confirmation email lands, closing the loop outside the system's own boundary. This is the definition of done. Not 'the function returned 200', but a real invitation in a real inbox — the outcome the entire architecture exists to produce.

7. Results & Demonstrated Capabilities

The system is live, serving a public custom domain over a global CDN, and reliably converts an anonymous click into either a verified booking or a specific, explained rejection.

7.1 Key Outcomes

- **A false-rejection rate reduced from 100% to zero** for legitimate authenticated visitors — the defining result of the rebuild.
- **162 of 162 policy combinations verified correct** by exhaustive enumeration.
- **A stateless serverless backend** with no idle cost and no infrastructure to patch.
- **Cryptographically signed, expiring approval** with constant-time verification.
- **Twelve granular rejection reasons** surfaced instead of a generic error.
- **A private-origin CDN architecture** in which the bucket is never publicly addressable.
- **Full request-level observability**, each decision reconstructable from logs alone.
- **A root cause found and fixed from evidence**, with the reasoning documented.

7.2 Operational Characteristics

Property	Result
Idle cost	Zero — no always-on compute in the architecture.
Cold-start headroom	30 s timeout against a multi-hop external identity handshake.
Memory headroom	512 MB configured against ~103 MB observed peak.
Public attack surface	One route: GET /callback.
Origin exposure	None — bucket private, reachable only via the CDN service principal.
Credential lifetime	15-minute signed token, unique replay identifier per issuance.
Policy change cost	Two configuration lines; no logic modification.
Auditability	Structured JSON decision record per request.

8. Security & Responsible Engineering

Because this system exists to make a security decision and was always intended to be published, security was treated as a design input rather than a review stage.

- **Federated identity over homegrown authentication.** No passwords are collected, stored or transmitted.

- **CSRF protection.** A cryptographically random nonce per attempt, persisted client-side and echoed through the round trip.
- **Server-side adjudication only.** Every check runs inside the function; the browser is never trusted with any part of the decision.
- **Single-use code semantics respected.** Reused codes are detected and classified as a retry condition, not misreported as verification failure.
- **Private origin by design.** Origin Access Control conditioned on the distribution ARN; the bucket is never publicly readable.
- **Least-privilege execution.** The role carries log-write permissions and no standing data-plane access.
- **Cryptographic hygiene.** HMAC-SHA256, explicit expiry, constant-time comparison, per-token replay identifiers.
- **Fail-closed defaults.** Any unhandled exception redirects to the failure page; no code path grants access on error.
- **Structured audit logging.** Policy, per-test state, data availability and verdict recorded for every request.
- **Safe-to-publish redaction.** Every account number, identifier, endpoint, bucket name, secret, signing key and token has been permanently redacted from all 56 captures, with the underlying credentials additionally rotated.

8.1 Disclosed Limitations

Responsible engineering means publishing the gaps alongside the controls. First, the approval token's verification on the static success page is cosmetic, and the underlying booking URL remains publicly reachable — the gate raises the cost of abuse without being an absolute boundary; the remediation is provider-minted single-use links generated inside the function. Second, secrets currently live as module constants; migrating them to a managed secret store with automatic rotation is the documented next step.

I identified both of these limitations myself, in my own documentation, before anyone else had the chance to.

9. The AI Avatar Video Resume

Embedded in my portfolio is a video resume presented by a synthetic avatar built on my own likeness and voice. It introduces my background, technical skills and the value I bring to clients and employers — and it does so as a generated artifact rather than a recorded one.

This is a deliberate piece of engineering rhetoric. A written claim of AI expertise is a claim; a working synthetic presenter that a visitor watches within seconds of landing on my site is **evidence**. The medium and the message are the same thing: the visitor is not told that I build applied AI systems, they are

shown one, immediately.

The introduction is not a description of the capability. It *is* the capability, running, in the visitor's browser.

It also demonstrates practical judgement about generative media. The avatar is built on my own likeness and voice with my full consent and clear provenance — a transparent use of synthetic media, precisely the responsible pattern organisations need modelled as this technology enters mainstream corporate communication. I handled delivery as an engineering problem too: the asset is served from the private origin through the CDN with a poster frame and metadata-only preload, so it never delays first paint for a visitor arriving on my site.

Taken together with the verification gateway, my portfolio makes a single coherent argument: **I architect the secure cloud infrastructure and I also build the applied AI that runs on top of it** — and both are demonstrably shipped, not described.

10. Skills Demonstrated

Domain	Demonstrated Skills
Cloud Architecture	Serverless event-driven design, managed API front doors, private-origin CDN distribution, scale-to-zero cost engineering, region and stage management.
AWS Engineering	Lambda (Node.js ESM, memory/timeout tuning), API Gateway HTTP APIs with proxy integration, S3 static origins, CloudFront distributions and invalidations, Origin Access Control, IAM policy authoring, CloudWatch Logs.
Security Engineering	OAuth 2.0 / OpenID Connect authorization-code flow, CSRF nonce protection, HMAC-SHA256 token design, constant-time comparison, replay identifiers, least privilege, fail-closed design, credential redaction and rotation, honest limitation disclosure.
Backend Development	Node.js ESM, asynchronous HTTP orchestration with parallel calls, defensive external-API handling, structured error taxonomy, configuration-driven policy engines.
Systematic Debugging	Multi-layer evidence gathering across rendered output, browser DOM console and cloud telemetry; timeout and memory diagnosis from execution reports; root-cause correction with documented justification.
Testing & Verification	Exhaustive state-space enumeration (162/162), three-valued logic modelling, specification-based validation.
Front-End Development	Hand-authored responsive HTML5/CSS3/JavaScript with no framework or build step, accessible media delivery, smooth-scroll navigation and active-state routing.
Applied AI	Generative avatar video production on my own likeness and voice, transparent synthetic-media provenance, integration of AI artifacts into a production web experience.

Domain	Demonstrated Skills
Technical Communication	Operations documentation with change-cost tables, failure-code references, troubleshooting runbooks and explicit maintainer warnings.

11. Conclusion & Future Enhancements

This project takes an ordinary requirement — a booking button on a personal website — and engineers it to a professional standard: federated identity, a stateless serverless decision engine, a configuration-driven policy validated across its entire input space, signed and expiring credentials, private-origin global delivery, structured observability, and documentation candid enough to name its own remaining gaps.

11.1 What This Project Demonstrates

A portfolio piece is only worth the capabilities a reader can verify from it. Everything claimed below is evidenced by something in this document — a screenshot, a configuration value, a decision, or a mistake and its correction.

Cloud architecture and AWS engineering. The system integrates six AWS services into a coherent whole: Lambda, API Gateway, S3, CloudFront, IAM and CloudWatch. That means selecting the right service tier for the job, sizing compute against measured evidence, and understanding the non-obvious couplings — that a region choice becomes part of a hostname registered with a third party, that a stage name silently reshapes every redirect URI, that configured memory governs CPU allocation. *Why this matters to you:* architecture is where cloud money and cloud outages are decided, and these are the couplings that cause both.

Security engineering. A full OAuth 2.0 / OpenID Connect authorization-code flow with CSRF defence, HMAC-SHA256 signed credentials with explicit expiry and constant-time verification, least-privilege IAM, private-origin storage scoped to a single distribution ARN, and fail-closed behaviour on every error path. *Why this matters to you:* most breaches are not exotic — they are a permission scoped too broadly or a boundary assumed rather than verified. This project shows both being handled deliberately.

Systematic debugging under uncertainty. A total failure was diagnosed from a generic HTTP 500 by working through three escalating layers of evidence to telemetry that proved the cause. No guessing, no shotgun fixes. *Why this matters to you:* this is the single most expensive skill gap on most teams. Engineers who guess extend outages; engineers who instrument end them.

Correcting the model rather than the symptom. The gate did not fail because of a typo. It failed because it conflated 'the provider will not tell us' with 'the visitor failed'. The fix was a redesign around three-valued logic. *Why this matters to you:* the ability to recognise a conceptual defect behind a technical symptom is the difference between a patch that holds and a bug that returns.

Verification and testing rigour. The policy was proven across all 162 valid combinations of its input space rather than spot-checked. *Why this matters to you:* when a system makes an access decision, 'it

worked when I tried it' is not evidence. Exhaustive proof where the space permits it is.

Applied AI delivery. A generated avatar video resume, produced on my own likeness and voice with transparent provenance and engineered for performant delivery. *Why this matters to you:* generative AI is moving into corporate communication, and the people worth hiring are those who can both build with it and model its responsible, consent-based use.

Technical communication. Operations notes with change-cost tables, a failure-code reference, troubleshooting runbooks, explicit warnings to future maintainers, and a published account of the system's own remaining weaknesses. *Why this matters to you:* undocumented systems become the property of whoever last touched them. Documented ones become assets the whole organisation can own.

Taken together, these are the capabilities of an engineer who can be handed an ambiguous problem and trusted to return a deployed, defensible, maintainable system — and to tell you the truth about what it does and does not do.

11.2 Planned Enhancements

- **Single-use booking links.** Mint provider-side one-time links inside the function so an approval cannot be shared or replayed.
- **Managed secret storage with rotation.** Migrate client credentials and the signing key out of module constants.
- **Elevated identity claims.** On grant of extended access, promote the badge and headline tests from scored to mandatory — a change the three-state design anticipates.
- **Server-rendered outcome pages.** Move token verification server-side so approval is enforced rather than presented.
- **Rate limiting and anomaly detection.** Throttle at the API surface and alert on unusual rejection patterns.
- **A verification analytics dashboard.** Aggregate decision records into trends on approval rates, failure reasons and data-source availability.
- **Infrastructure as code.** Express the full stack declaratively for reproducible, reviewable deployment.
- **Expanded applied-AI surface.** Extend the avatar work toward an interactive, retrieval-grounded assistant answering visitor questions directly on the site.

12. Let's Build Something

I design and build systems like this one end to end — architecture, implementation, deployment, debugging, security review and the documentation that makes them maintainable. Cloud solution architecture on AWS and Azure, applied machine learning and agentic AI automation, full-stack and cloud-native application development, data engineering and analytics, and the technical and strategic consulting that connects them to business outcomes.

I bring a U.S. Air Force veteran's operational discipline, a Master of Science in Computer Science, and a track record of taking ambiguous problems all the way to deployed, documented, defensible systems. If your organisation needs someone who can architect it, build it, prove it works, and explain it clearly to both engineers and executives — I would welcome the conversation.

Portfolio	esmeraldaspace.com
LinkedIn	linkedin.com/in/esmeraldaspace
GitHub	github.com/esmeralda-cabrera-ventura
Schedule a conversation	Use the verified Schedule an Interview flow documented in this report — it is live on the site right now.

Designed, engineered, deployed, debugged and documented by Esmeralda Cabrera Ventura.